# Sri Lanka Institute of Information Technology




**Continuous Assessment**

DevOps Security | IE3142

Y3.S1.WE.CS.02

| IT Number | Name |
|---|---|
| IT24102747 | Kosgollage S R |
| IT24201195 | Palugaswewa K I K I B |
| IT24102899 | Shaheen M F M |
| IT24102514 | Aaqib H M |

# Executive summary & System Overview

This project focuses on designing, implementing, and automating a comprehensive DevSecOps pipeline for OWASP NodeGoat, an intentionally vulnerable web application. The main purpose of this project is to transform a vulnerable application into an enterprise-level secure software. The project addresses the primary risks associated with insecure applications by focusing on application security through the implementation of a pipeline that automatically scans, tests, and verifies code changes to protect the application from attacks. The end result of this project is the creation of an enterprise-level secure application with a STRIDE-based threat model, a comprehensive GitHub Actions workflow, and a complete containerized platform.

[1] [10]

## System Overview

The application chosen for this DevSecOps implementation is OWASP NodeGoat. It is primarily used because of its unique two-tiered component structure and the fact that it includes common real-world flaws that are present in actual industry applications. Additionally, this application is not reliant on live third-party cloud services, making it ideal for the requirements of this project, in which isolation and the use of open-source solutions are prioritized.

**Technology Stack**

- **Application Tier:** Built using Node.js and the Express.js framework, serving both the web interface and backend API logic.
- **Data Tier:** Utilizes MongoDB, a NoSQL database, to manage user credentials, session storage, and core application data.
- **Pipeline & Automation:** GitHub Actions serves as the CI/CD orchestrator, utilizing automated scanning tools (such as Semgrep, npm audit, Gitleaks, and Trivy) to form the security gates. [11] [12]

**Containerization approach**

To guarantee environmental consistency and streamline deployment, both architectural components were explicitly containerized. The Node.js web server and the MongoDB database are packaged into isolated Docker images using separate Dockerfiles. Orchestration is managed via a docker-compose.yml configuration, which bridges the two containers over an isolated internal Docker network. This deployment strategy allows the complete environment to be initialized with a single command while enforcing strict trust boundaries - ensuring the database

**Architecture Diagram:**

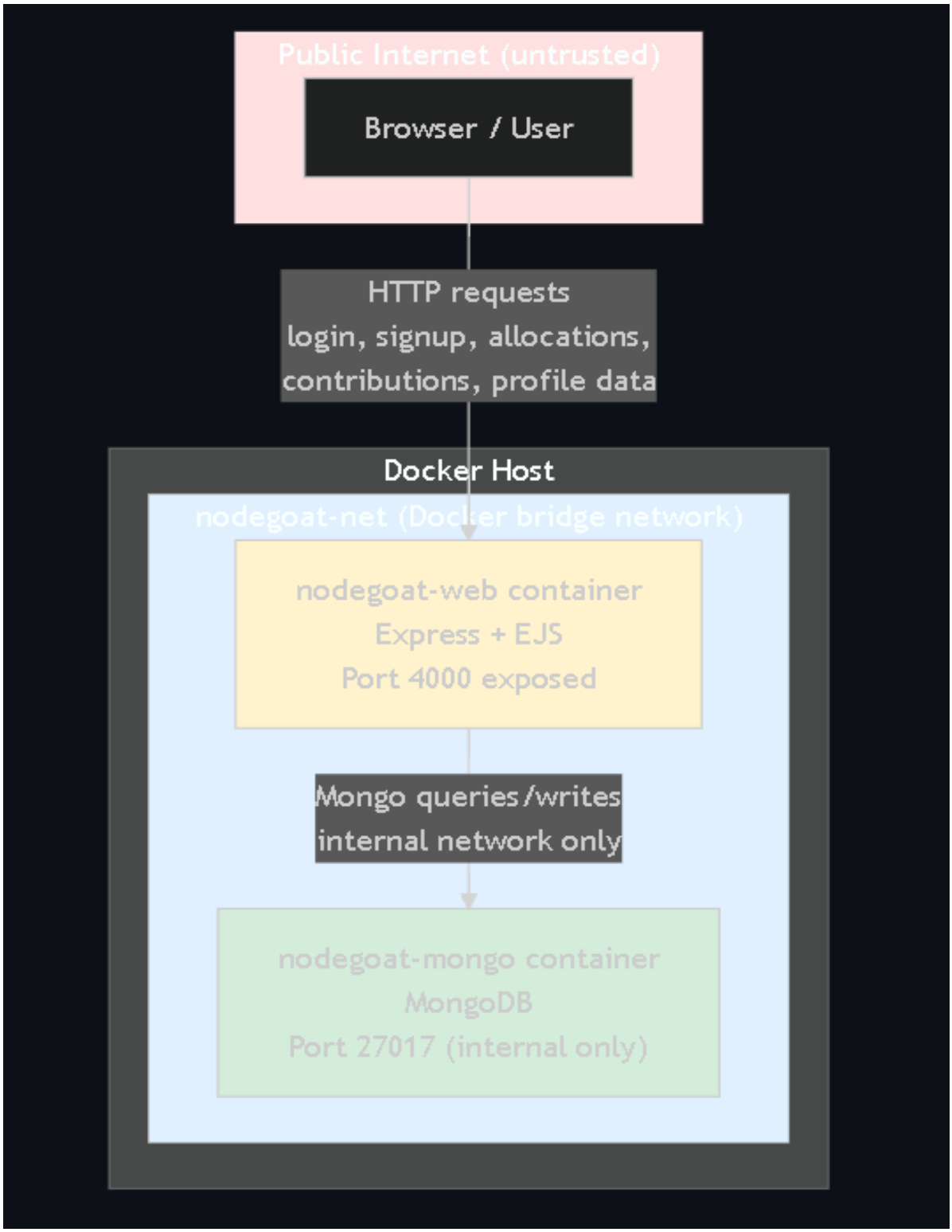



*Figure 1.Architecture Diagram*

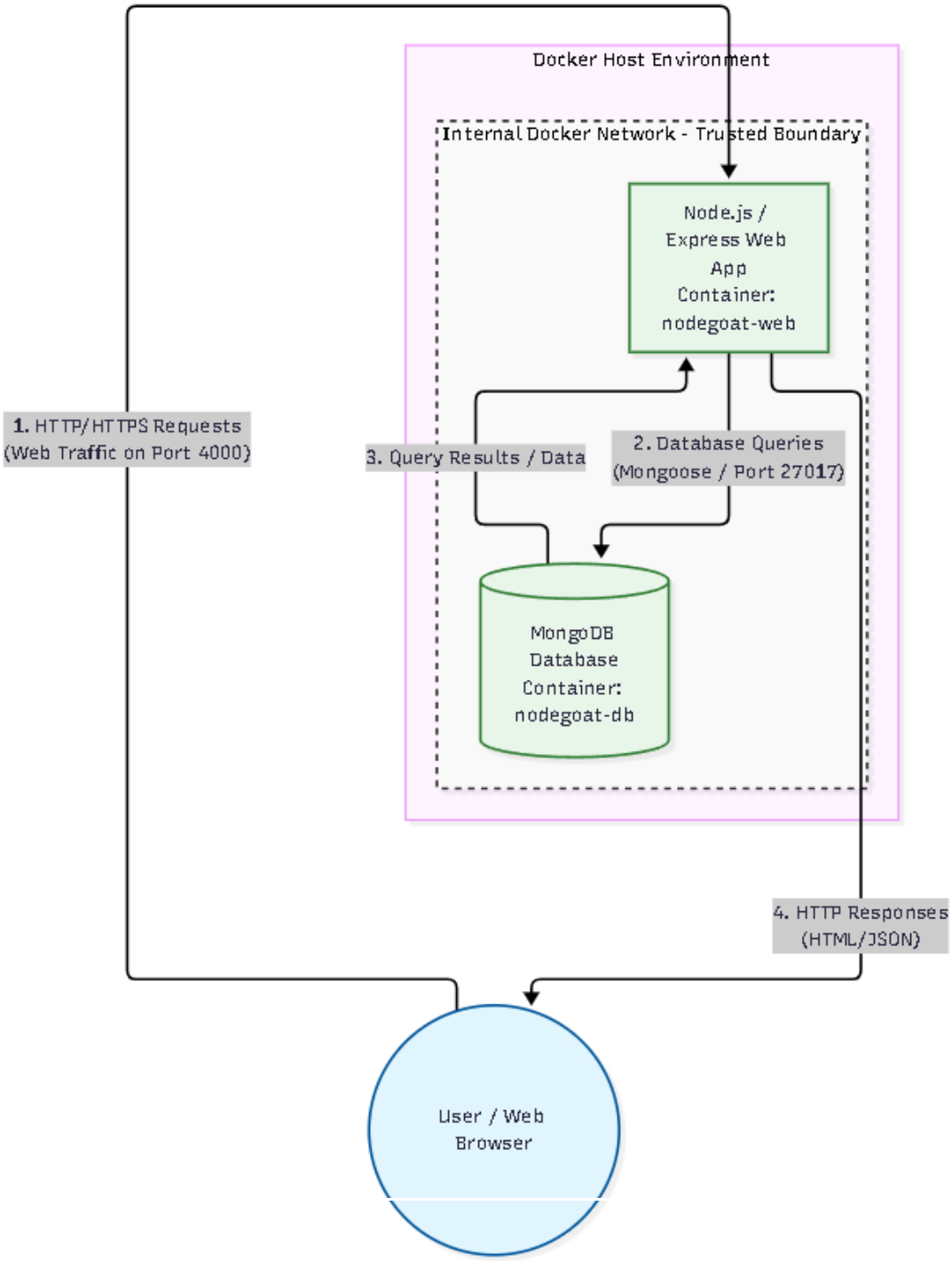



*Figure 1 - dataflow diagram*

## Threat modelling and risk assessment

The architecture has three trust boundaries: (1) the Internet and the user's browser, which are untrusted; (2) the web container, which processes untrusted input; and (3) the internal Docker network containing MongoDB, which holds all persistent data. A fourth boundary covers the CI/CD path, where GitHub, the Actions runner and the stored secrets act as a separate trust zone. Threats were identified by applying STRIDE to each data flow crossing these boundaries. Likelihood and impact are scored 1–5 and multiplied to give a risk score (1–25); scores of 15 or more are treated as high. [2]

| # | **Stride** | **Threat** | **Likelihood** | **Impact** | **Justification** | **Control** | **Location** |
|---|---|---|---|---|---|---|---|
| 1 | Tampering | NoSQL/JavaScript injection: the threshold query parameter of /allocations/:userId is interpolated into a MongoDB $where string (CWE-943) [3] | High | High | Any logged-in user controls the parameter and no tooling is needed; injected JavaScript runs inside MongoDB and can read other users' data | Validate threshold as an integer and replace $where with a standard query using $gt | app/data/allocations-dao.js (lines 73, 78) |
| 2 | Elevation of privilege | Server-side code injection: preTax, afterTax and roth are passed to eval() (CWE-95)[4] | High | High | Signups are open, so any visitor can reach the form; eval gives arbitrary JavaScript execution in the Node process, including access to its environment variable | Replace eval() with parseInt() and a range check on each field | app/routes/contributions.js (lines 32-34) |
| 3 | Spoofing | Session fixation: login does not regenerate the session ID, so a pre-login session ID stays valid afterwards (CWE-384); distinct username/password errors allow user enumeration[5], [6] | Medium | High | The attacker must plant a session ID in the victim's browser, which is harder than injection, but success gives full account takeover | Call req.session.regenerate() on login, return one generic error message, and load the cookie secret from an environment variable | app/routes/session.js, config/env/all.js |
| 4 | Stored(XSS) | swig autoescape is | High | Medium | Any user can store a script | Set autoescape: true and add a | server.js (lines 135-137), app/views/*.html |

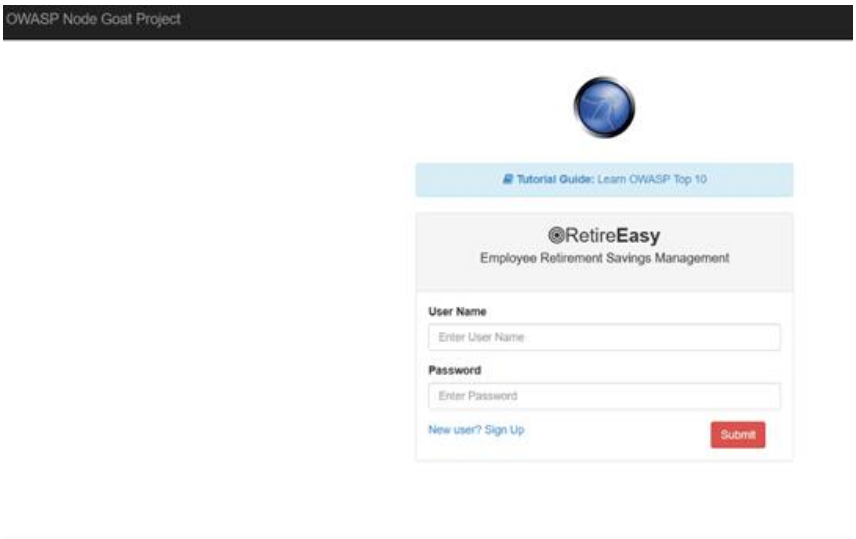
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | disabled, so user-supplied content is rendered as raw HTML to other users (CWE-79)[7], [8] | | | payload; impact is limited to the victim's browser session, not the server | Content-Security-Policy via helmet | |
| 5 | Information disclosure | IDOR: /allocations/:userId trusts the ID in the URL instead of the logged-in user (CWE-639)[9] | High | Medium | User IDs appear to be sequential integers, so guessing is trivial; exposure is limited to other users' allocation data | Take userId from req.session and ignore the URL parameter | app/routes/allocations.js |

# Vulnerability assessment and secure coding evidence

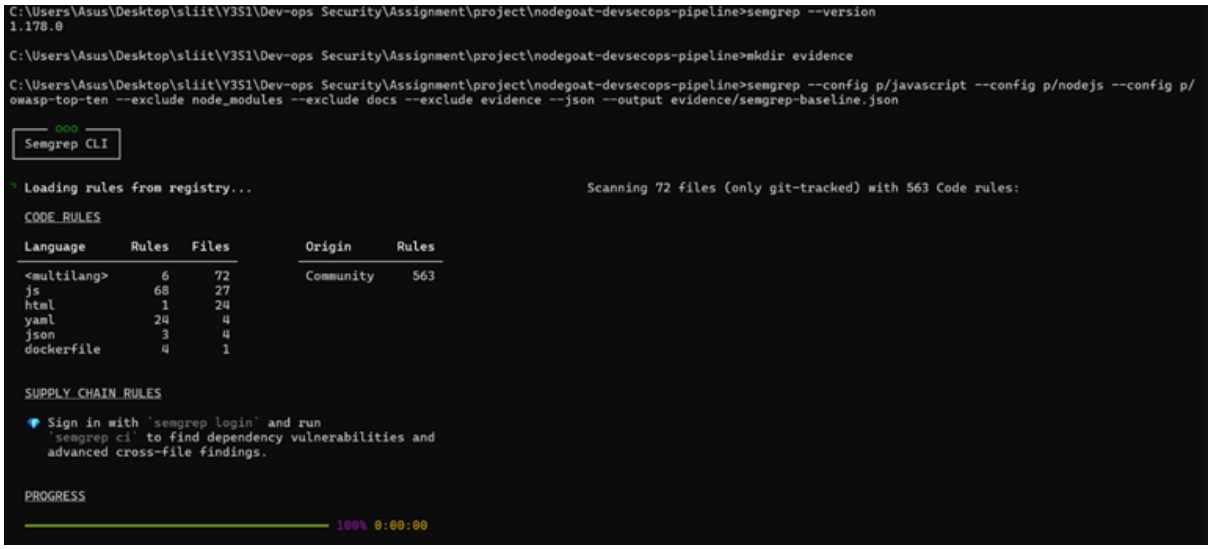
Vulnerabilities are clearly identified and analyzed; at least two fixes are well implemented with strong before/after and rescan evidence

## Vulnerability Scan Evidence

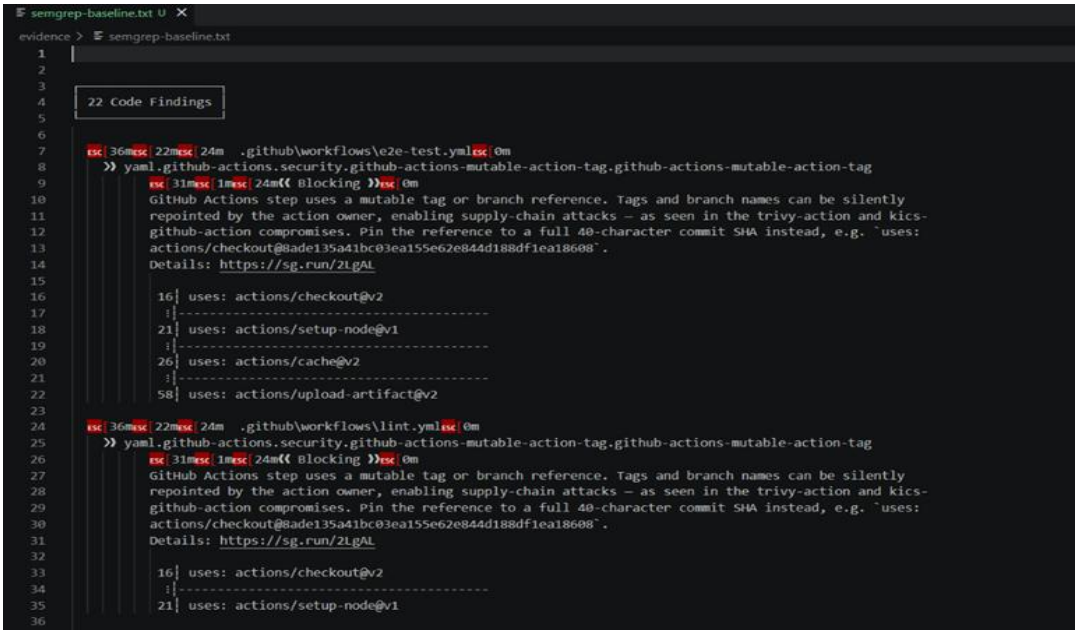
The supplied evidence begins with vulnerability-scanning output and supporting application/code views. These screenshots establish the starting point for the subsequent manual exploit demonstrations and remediation checks.



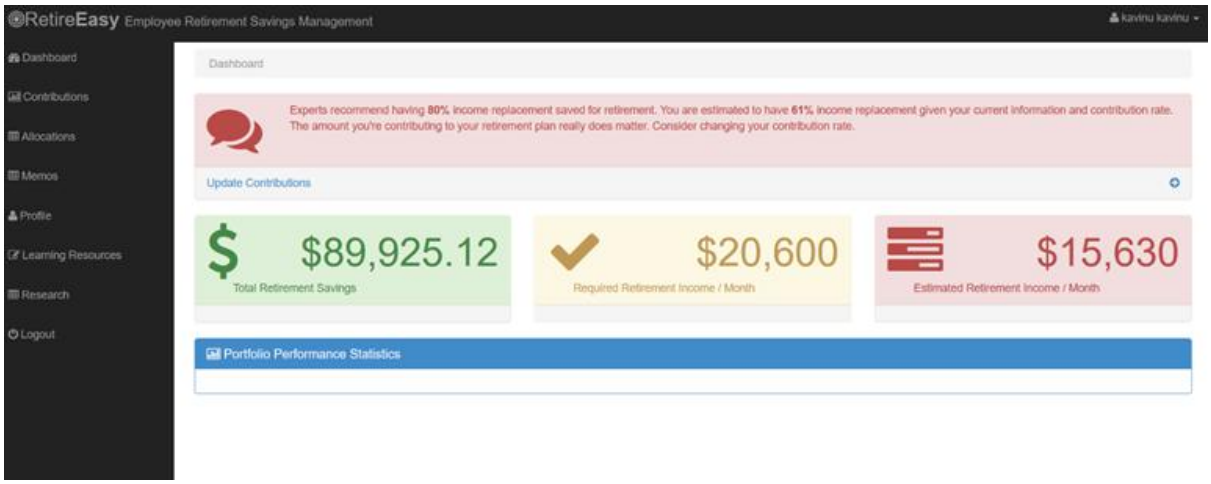

Evidence 1 -Application/login interface used during testing. [File: image1.png]




Evidence 4 -Supporting code or configuration evidence. [File: image4.png]




Evidence 8 -Code or security-analysis evidence. [File: image8.png]




Evidence 9 -Application/dashboard evidence. [File: image9.png]

# 1. Vulnerability 1 -NoSQL Injection

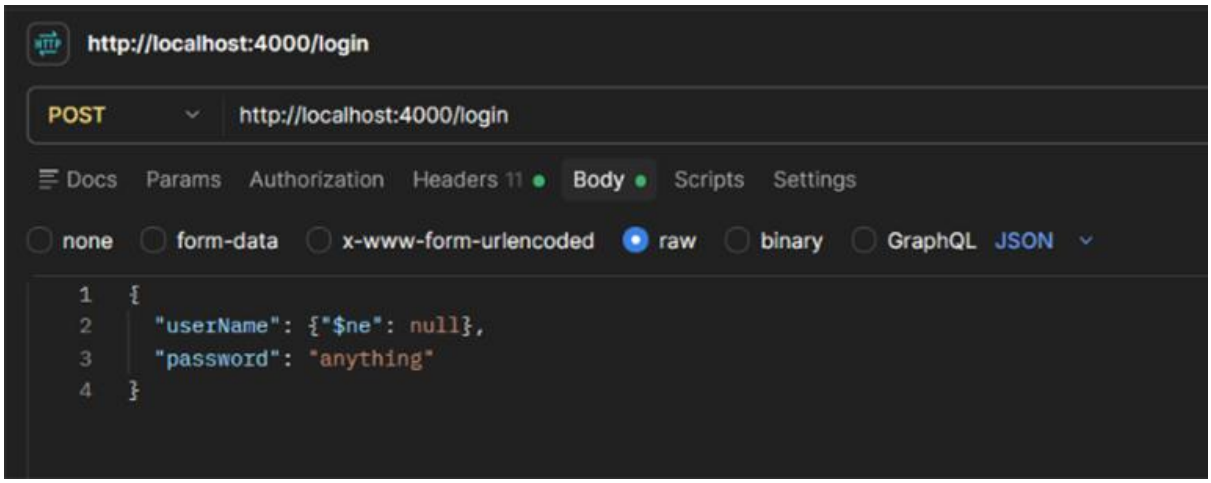
The demonstration identifies a NoSQL injection vulnerability in the username field. The reported root cause is that userName from the request body is passed directly into a MongoDB query without first validating that it is a plain string.

## *1.1 Exploit demonstration*

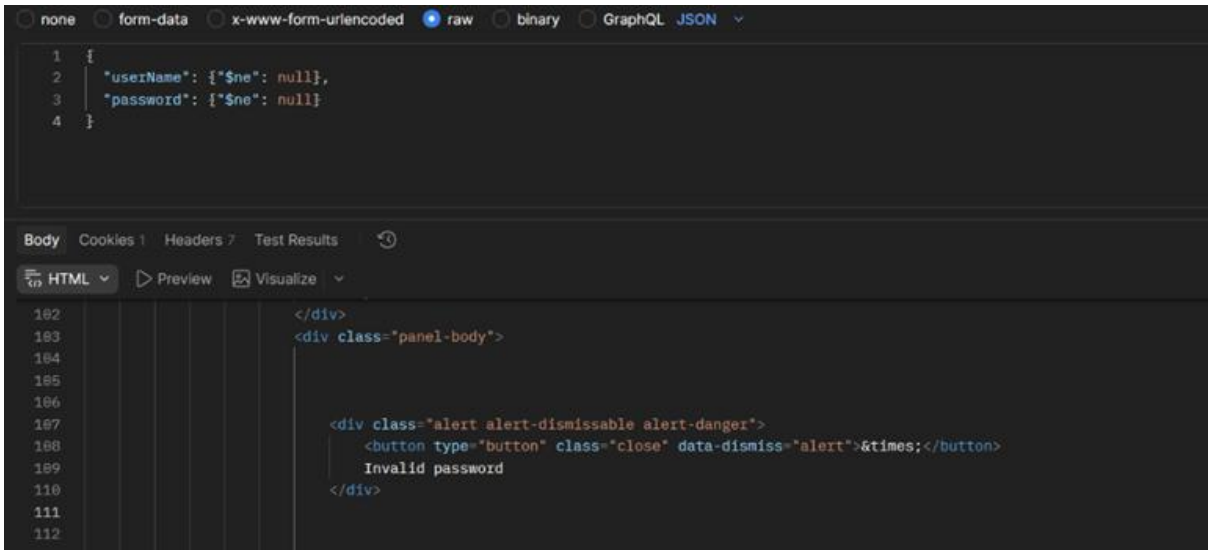
The exploit evidence shows the vulnerable input handling and demonstrates that the injection point is isolated to the username field. This supports the stated threat-model scope.




Evidence 10 -Request or application evidence showing the vulnerable login interaction. [File: image10.png]




Evidence 11 -Supporting request/parameter evidence. [File: image11.png]




Evidence 12 -Code or request evidence associated with the injection point. [File: image12.png]

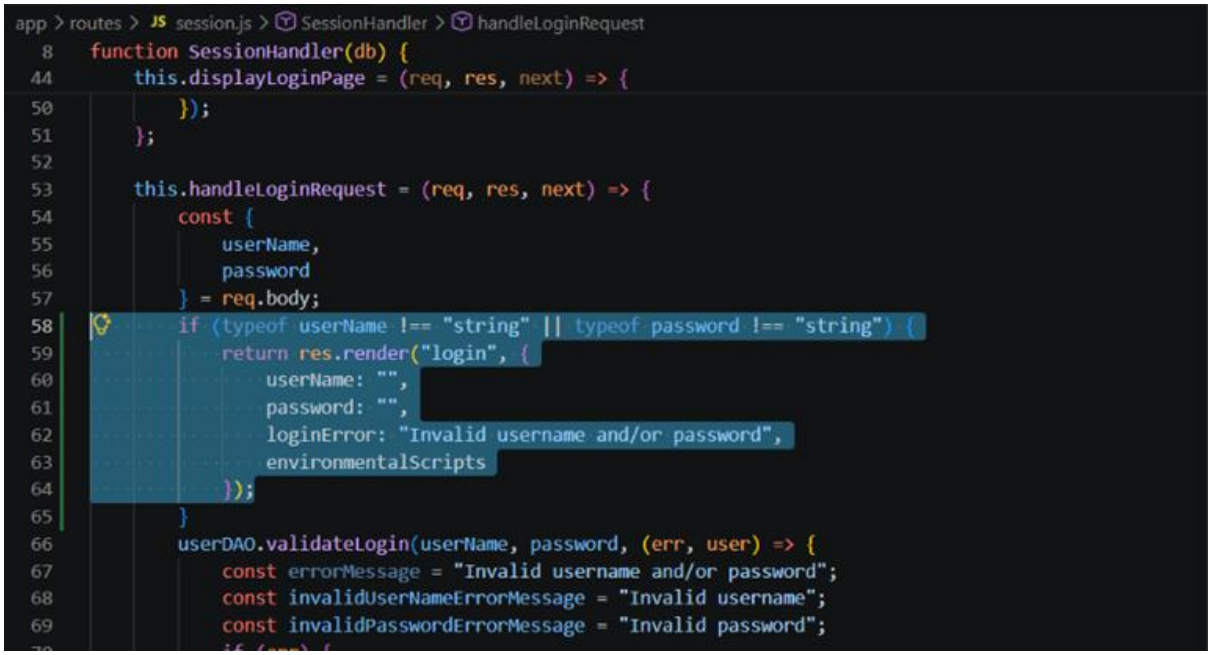



Evidence 13 -Application response demonstrating the test input condition. [File: image13.png]

The evidence is used to establish that the username field is the relevant injection point rather than treating every login parameter as vulnerable.

### *1.2 Remediation*

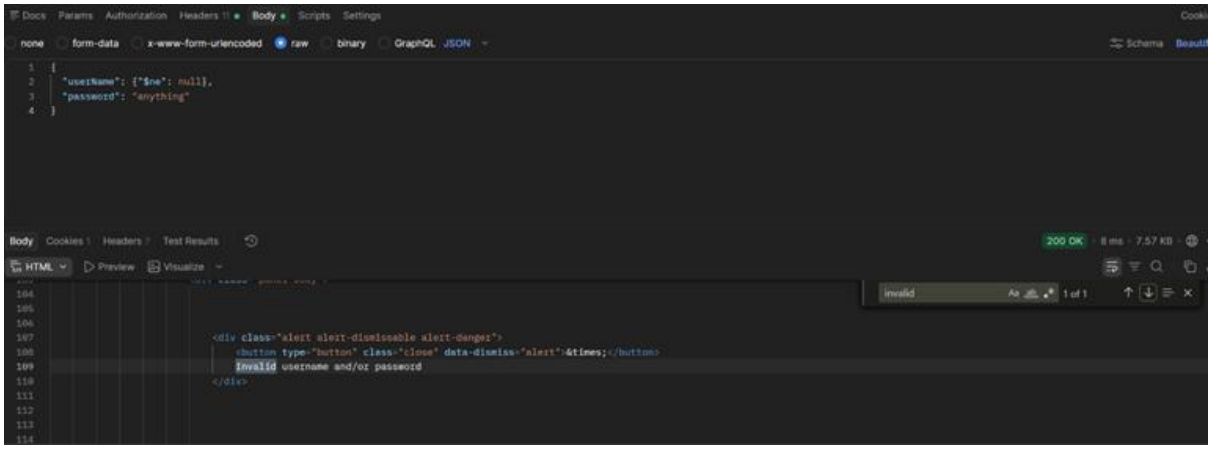
The documented fix is input type validation. The application should reject any value that is not a plain string before that value reaches the MongoDB query.




Evidence 14 -Code showing the type-validation guard added for userName. [File: image14.png]

### *1.3 Verification after the fix*

After the fix, the type-check is present in the request-processing path. The supplied document also compares the post-fix state with the earlier behavior.




Evidence 15 -Post-fix application or test evidence. [File: image15.png]




Evidence 16 -Supporting post-fix result. [File: image16.png]

The document notes that Semgrep did not change its finding count because the fix is a targeted runtime logic change rather than one of the generic syntactic patterns covered by the relevant Semgrep rules. The existing findings were described as unrelated issues elsewhere in the application.

## 2. Vulnerability 2 -Weak Session Management and Session Fixation[5], [6]

The document identifies two related weaknesses: a hardcoded, predictable session secret and failure to rotate the session identifier after authentication. Together, these conditions allow the demonstrated session-fixation attack.

### *2.1 Attack concept*

- A predictable or known session secret allows an attacker to create a cookie with a chosen session identifier and a valid signature.
- The attacker causes the victim to use the attacker-selected session identifier before authentication.
- The victim logs in without the application generating a new session identifier.
- The authenticated session therefore remains associated with the identifier already known to the attacker.

- The attacker can then reuse the same session identifier to access the victim's authenticated session.

config > env > JS all.js > ...

```
// default app configuration
const port = process.env.PORT || 4000;
let db = process.env.MONGODB_URI || "mongodb://localhost:27017/nodegoat";

module.exports = {
    port,
    db,
    cookieSecret: "session_cookie_secret_key_here",
    cryptoKey: "a_secure_key_for_crypto_here",
    cryptoAlgo: "aes256",
    hostName: "localhost",
    environmentalScripts: []
};
```

Evidence 18 -Supporting code/configuration evidence for session handling. [File: image18.png]

JS forge-cookie.js U | all.js

JS forge-cookie.js > ...

```
// Demonstrates that NodeGoat's session cookie secret is hardcoded and
// publicly known, allowing an attacker to forge a validly-signed session
// cookie for any session ID of their choosing (session fixation).
//
// Usage: node forge-cookie.js
const cookieSignature = require("cookie-signature");

// This value is taken verbatim from config/env/all.js — it is committed
// to source control and identical in every NodeGoat deployment that uses
// the default config, making it public knowledge.
const KNOWN_SECRET = "session_cookie_secret_key_here";

// The attacker picks this session ID themselves, in advance.
const ATTACKER_CHOSEN_SESSION_ID = "attacker-fixed-session-12345";

const signed = cookieSignature.sign(ATTACKER_CHOSEN_SESSION_ID, KNOWN_SECRET);

// express-session cookies are stored URL-encoded with an "s:" prefix
// before the signed value.
const cookieValue = "s%3A" + encodeURIComponent(signed).replace(/%2C/g, ",");

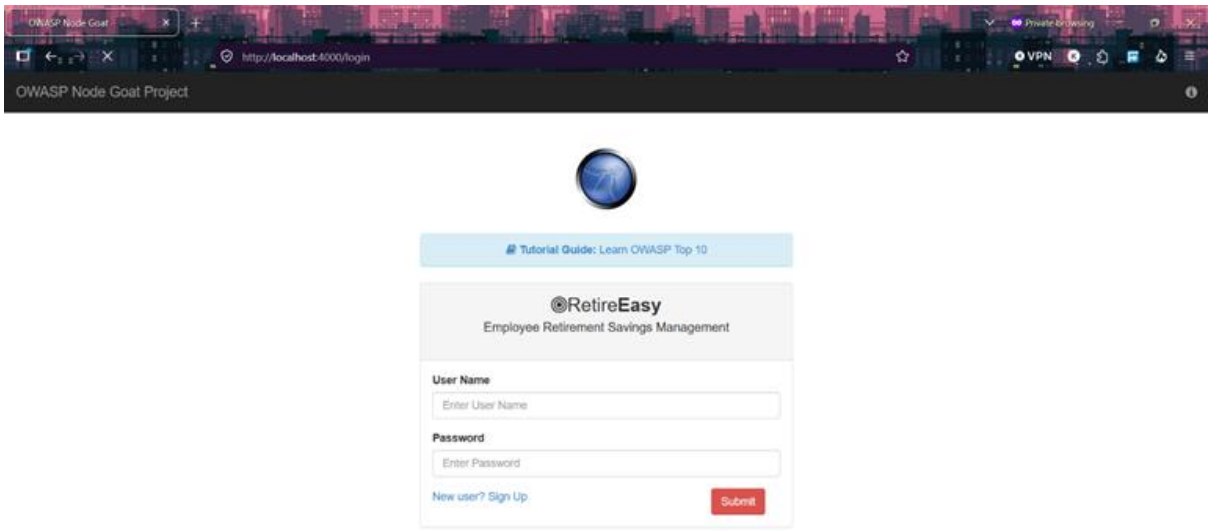
console.log("Attacker-chosen session ID:", ATTACKER_CHOSEN_SESSION_ID);
console.log("Forged signed cookie value (connect.sid):");
console.log(cookieValue);
console.log("");
console.log("To use: open browser dev tools on http://localhost:4000,");
console.log("go to Application/Storage > Cookies, and set connect.sid to");
console.log("the value above (or use document.cookie in the console).");
```

Evidence 19 -Supporting session-related code or configuration. [File: image19.png]

The document then demonstrates the attack in a controlled local test environment.

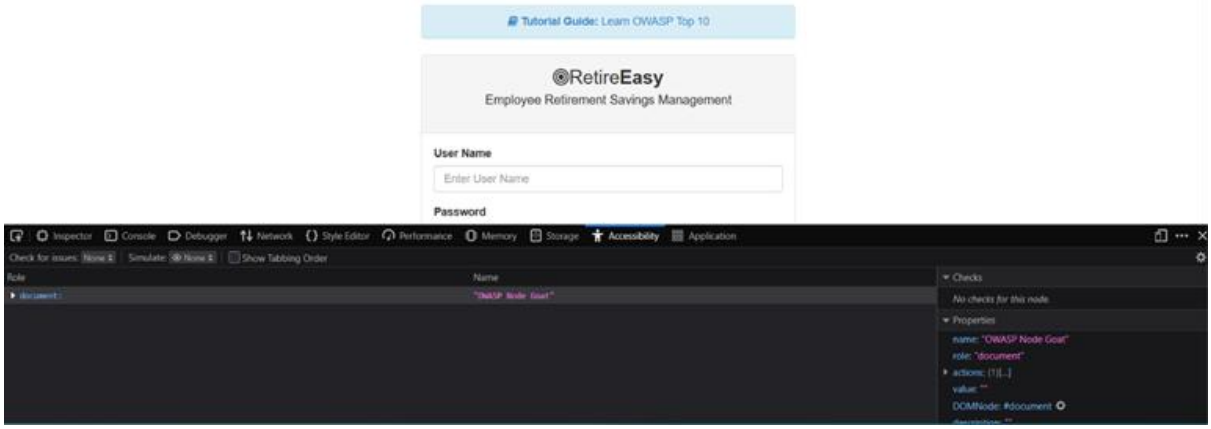
## *2.2 Initial browser-based demonstration*

Start with a clean browser session so that an existing authenticated session does not interfere with the test.




Evidence 20 -Fresh application session used for the demonstration. [File: image20.png]

Open Developer Tools and inspect the browser session/cookie state.



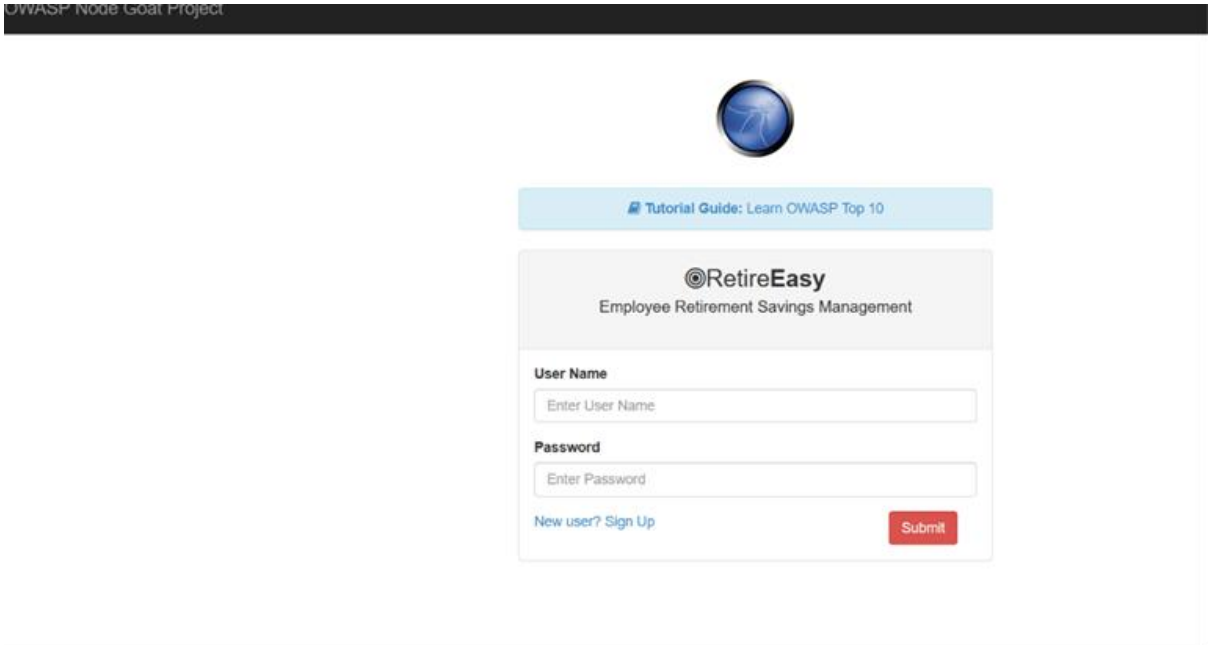

Evidence 21 -Developer Tools/session state before the forged-cookie test. [File: image21.png]

The document then uses the browser console to set a forged, attacker-selected session cookie.



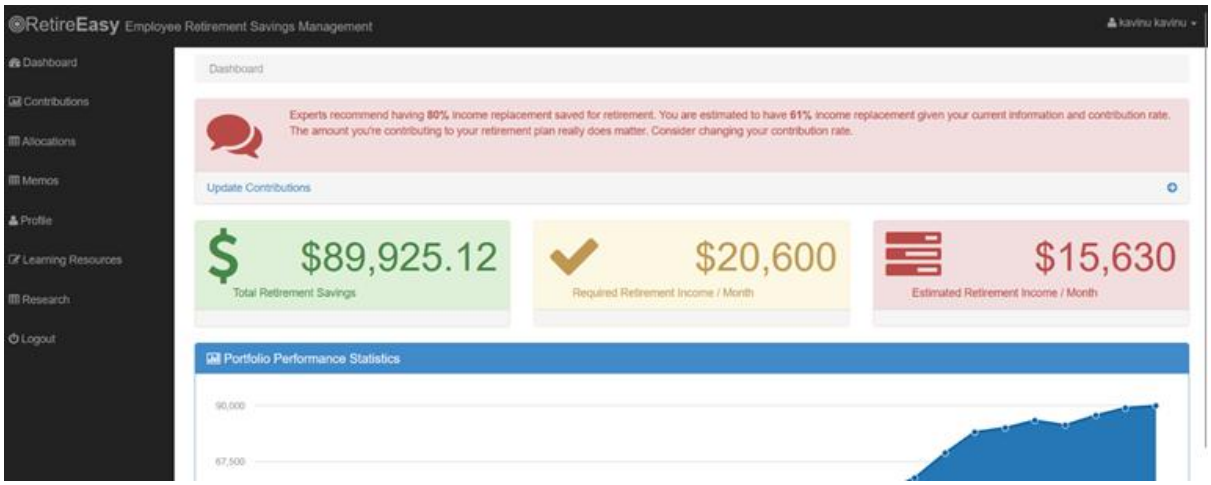

Evidence 22 -Console command used to set the attacker-selected session cookie. [File: image22.png]

After refreshing, the server accepts the cookie as correctly signed but does not associate the new session identifier with an authenticated user. The expected behavior is therefore a redirect to the login page rather than a signature error.

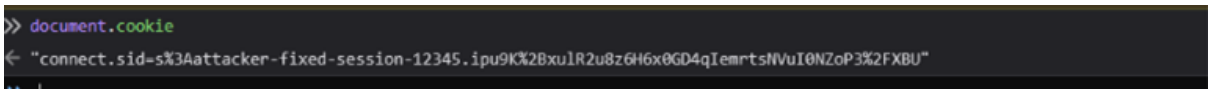



Evidence 23 -Result after presenting the forged session cookie. [File: image23.png]

The next step is to log in normally while retaining the same browser/session context.

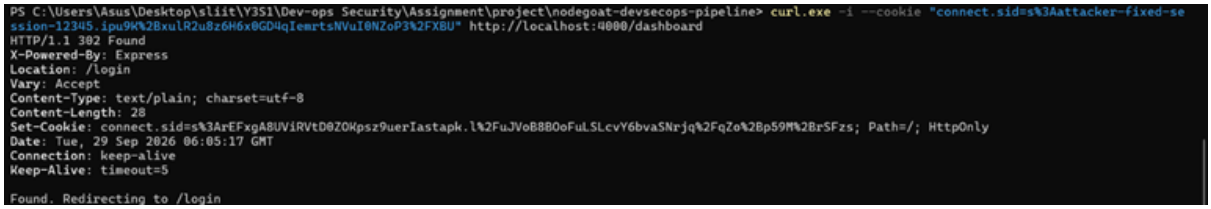



Evidence 24 -Login interaction while the test session is present. [File: image24.png]




Evidence 25 -Cookie/session value observed after the login interaction. [File: image25.png]

The document explains that the server verifies the cookie signature but, for the newly chosen identifier, finds no existing session data. In that browser-based test the server consequently creates a new session, which makes the browser view less suitable for proving fixation by itself.



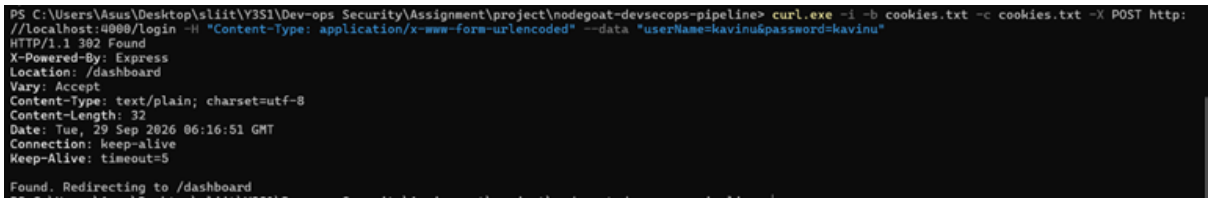
Evidence 26 -Command-line evidence showing the server response to the forged cookie.  [File: image26.png]

## *2.3 Controlled session-fixation proof*

The cleaner demonstration uses a real server-issued pre-authentication session and then reuses that exact session during login.

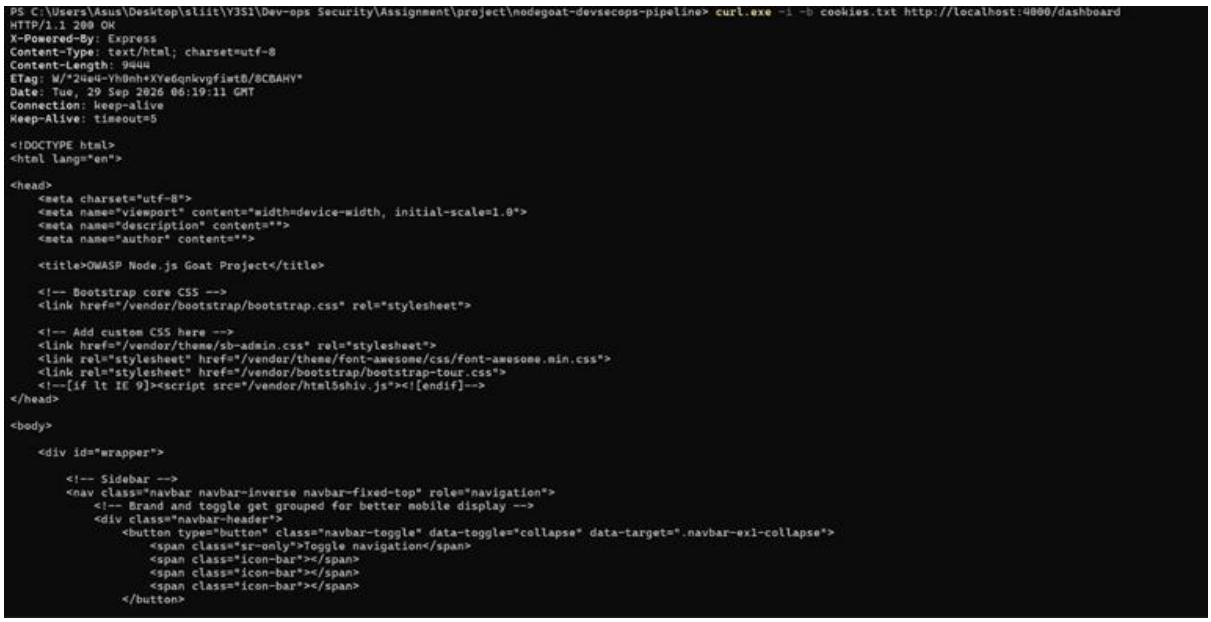



Evidence 27 -Step 1: real server-issued pre-authentication session.  [File: image27.png]




Evidence 28 -Step 2: login while presenting the same session.  [File: image28.png]

The supplied evidence states that no Set-Cookie header was returned during login. The same pre-authentication session identifier therefore remained associated with the authenticated account.




Evidence 29 -Supporting HTTP/session evidence for the unchanged session identifier.  [File: image29.png]




Evidence 30 -Supporting response/header evidence.  [File: image30.png]

```
        <i class="fa fa-check fa-5x"></i>
    </div>
    <div class="col-xs-11">
        <p class="announcement-heading text-right">$20,600</p>
        <p class="announcement-text">Required Retirement Income / Month</p>
    </div>
</div>
iv>
```

Evidence 31 -Supporting session evidence. [File: image31.png]

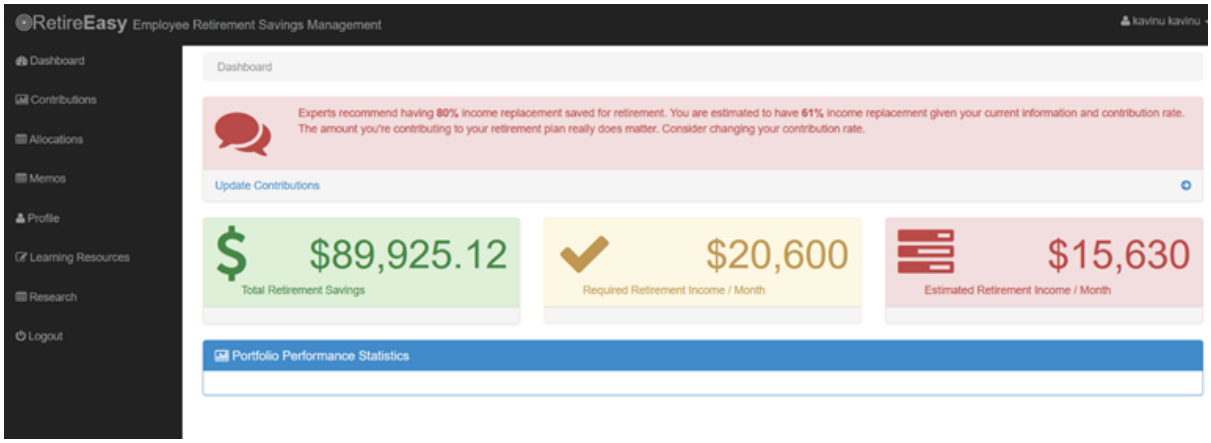
```
</div>
<div class="col-xs-11">
    <p class="announcement-heading text-right">$15,630</p>
    <p class="announcement-text">Estimated Retirement Income / Month</p>
</div>
iv>
```

Evidence 32 -Supporting session evidence. [File: image32.png]

```
            <div class="col-xs-1">
                <i class="fa fa-usd fa-5x"></i>
            </div>
            <div class="col-xs-11">
                <p class="announcement-heading text-right">$89,925.12</p>
                <p class="announcement-text">Total Retirement Savings</p>
            </div>
        </div>
    </div>
    <a href="#">
```

Evidence 33 -Supporting session evidence. [File: image33.png]

The final proof reuses the original cookies.txt value to access a protected page without supplying the username or password again.



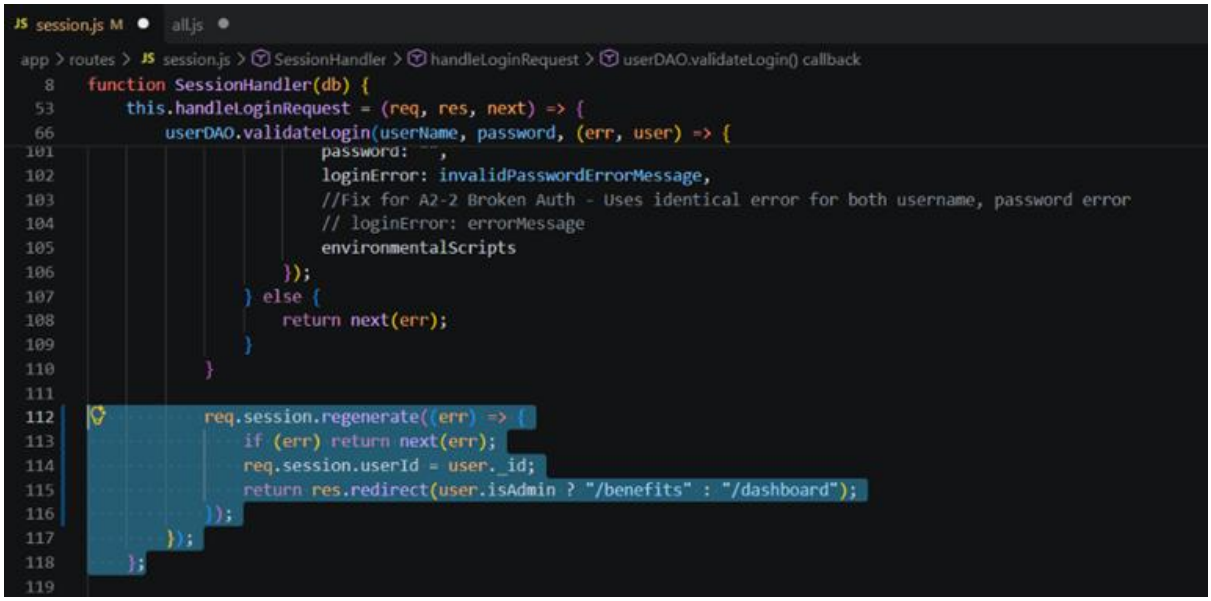

Evidence 34 -Protected dashboard returned using the original session cookie. [File: image34.png]

The document reports a successful HTTP 200 response containing the authenticated user's dashboard and account information. This is the evidence used to confirm the session-fixation hijack in the test environment.

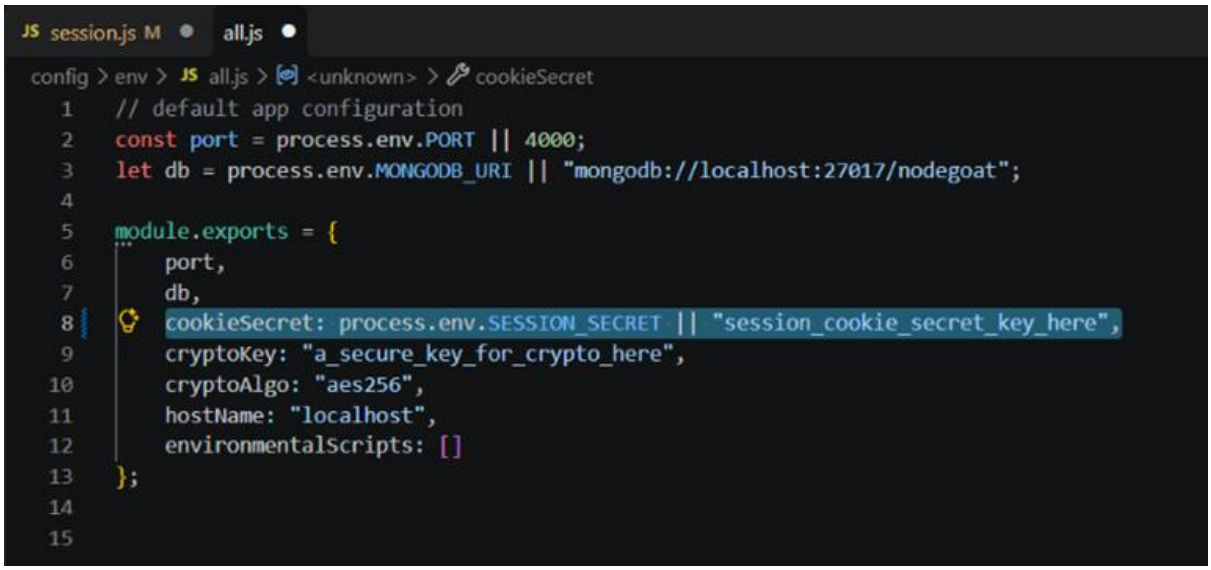
### *2.4 Remediation*

The first remediation is to regenerate the session identifier at login. Previously, the session identifier present before authentication was carried through unchanged. The fix makes login

issue a new identifier, invalidating an identifier that may have been planted before authentication.

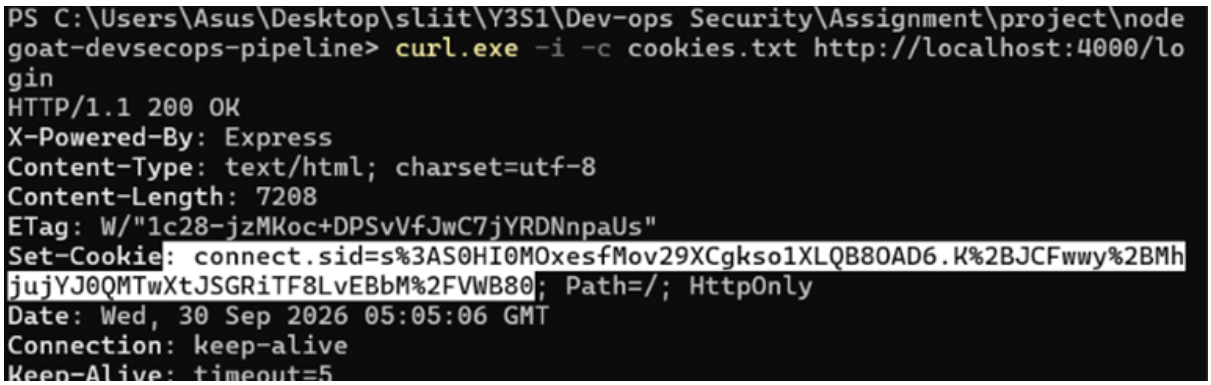



Evidence 35 -Code showing session-regeneration logic. [File: image35.png]




Evidence 36 -Supporting session-regeneration implementation. [File: image36.png]

The second remediation is to replace the hardcoded session secret with an environment variable.



Evidence 37 -Evidence of the updated session-secret handling. [File: image37.png]

```
PS C:\Users\Asus\Desktop\sliit\Y3S1\Dev-ops Security\Assignment\project\node
goat-devsecops-pipeline> curl.exe -i -b cookies.txt -c cookies.txt -X POST h
ttp://localhost:4000/login -H "Content-Type: application/x-www-form-urlencod
ed" --data "userName=kavinu&password=kavinu"
HTTP/1.1 302 Found
X-Powered-By: Express
Location: /dashboard
Vary: Accept
Content-Type: text/plain; charset=utf-8
Content-Length: 32
Set-Cookie: connect.sid=s%3AmluvSACONfjWUdFThZ5UhjYdSOUYSBY-.K2Edox9UyPXbMhN
KxwHx4H9P%2BdlD7Au%2BueUaTxqM7JE; Path=/; HttpOnly
Date: Wed, 30 Sep 2026 05:05:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Evidence 38 -Supporting configuration or code evidence. [File: image38.png]

### *2.5 Verification after the fix*

The post-fix evidence shows a pre-login session identifier followed by a different identifier after login, demonstrating that session.regenerate() is being invoked.

```
<!-- @FIXME use a properly escaped variable that matches the URL context, for example
refer to a firstNameSafeURLString field on the doc object set by the controller for this template -->
<a href="{{firstNameSafeString}}">Google search this profile by name</a>
```

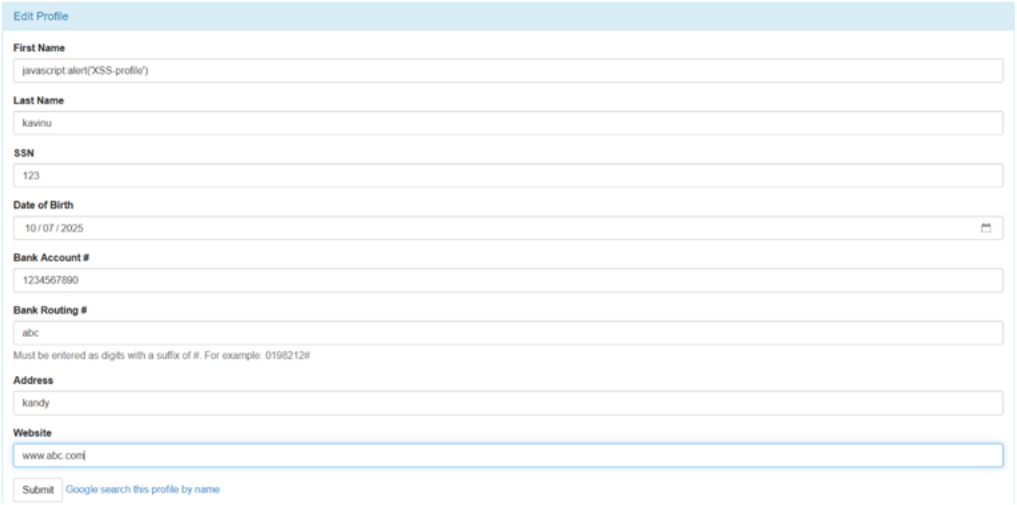
Evidence 39 -Post-fix session identifiers showing rotation. [File: image39.png]

The document concludes that the same attack that succeeded before the fix no longer succeeds after session regeneration and the secret-management change.

## 3. Vulnerability 3 -Stored Cross-Site Scripting (XSS)

The document identifies a stored XSS issue involving firstNameSafeString in the error-handling path of handleProfileUpdate. The value is assigned directly from firstName without encoding and is later rendered into an href attribute. The application-wide Swig configuration also disables automatic escaping.

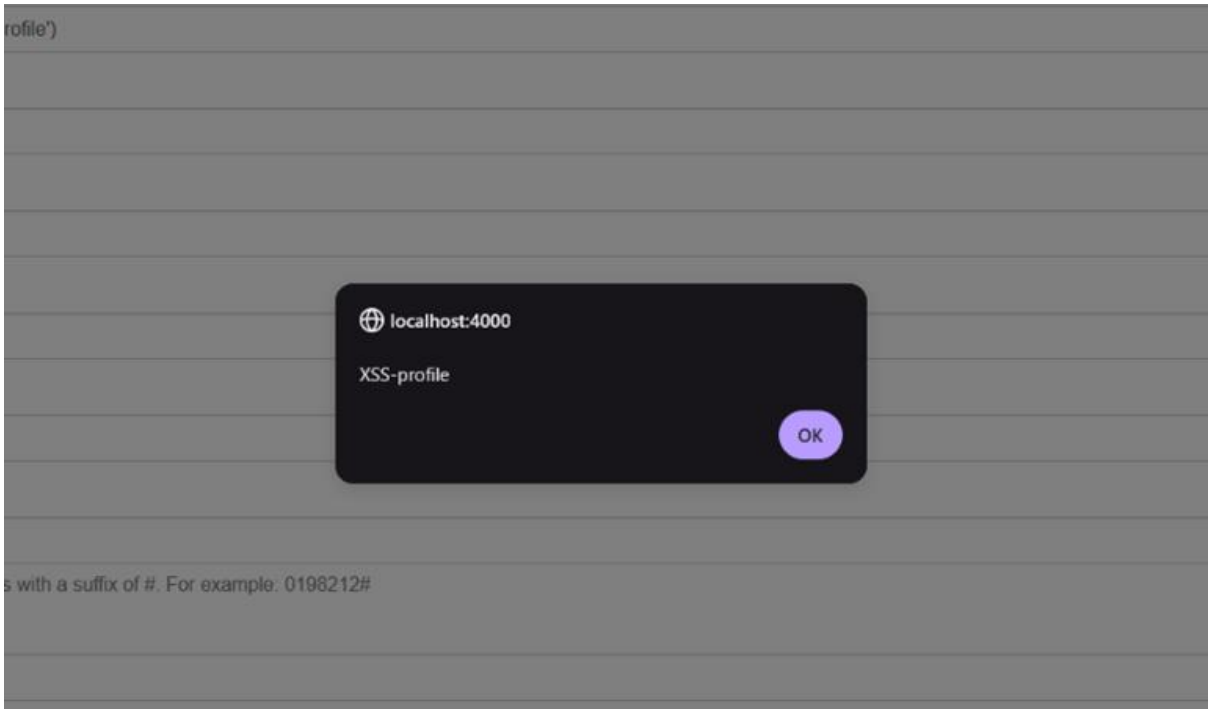
### *3.1 Vulnerable data flow*




Evidence 40 -Profile form/application state used during the XSS demonstration. [File: image40.png]

The vulnerable value can reach the profile page's 'Google search this profile by name' link. The demonstrated payload is:

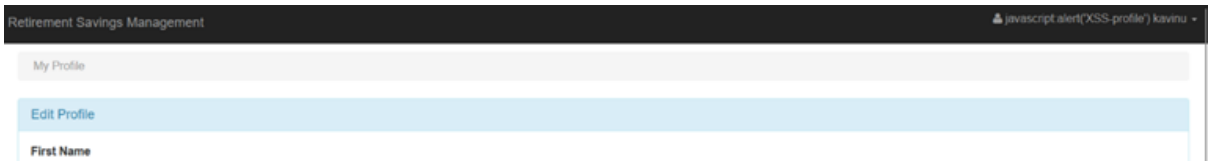
```
javascript:alert('XSS-profile')
```




Evidence 41 -Demonstrated XSS execution result. [File: image41.png]




Evidence 42 -Profile page showing the affected value/link. [File: image42.png]




Evidence 43 -Supporting URL/link evidence. [File: image43.png]

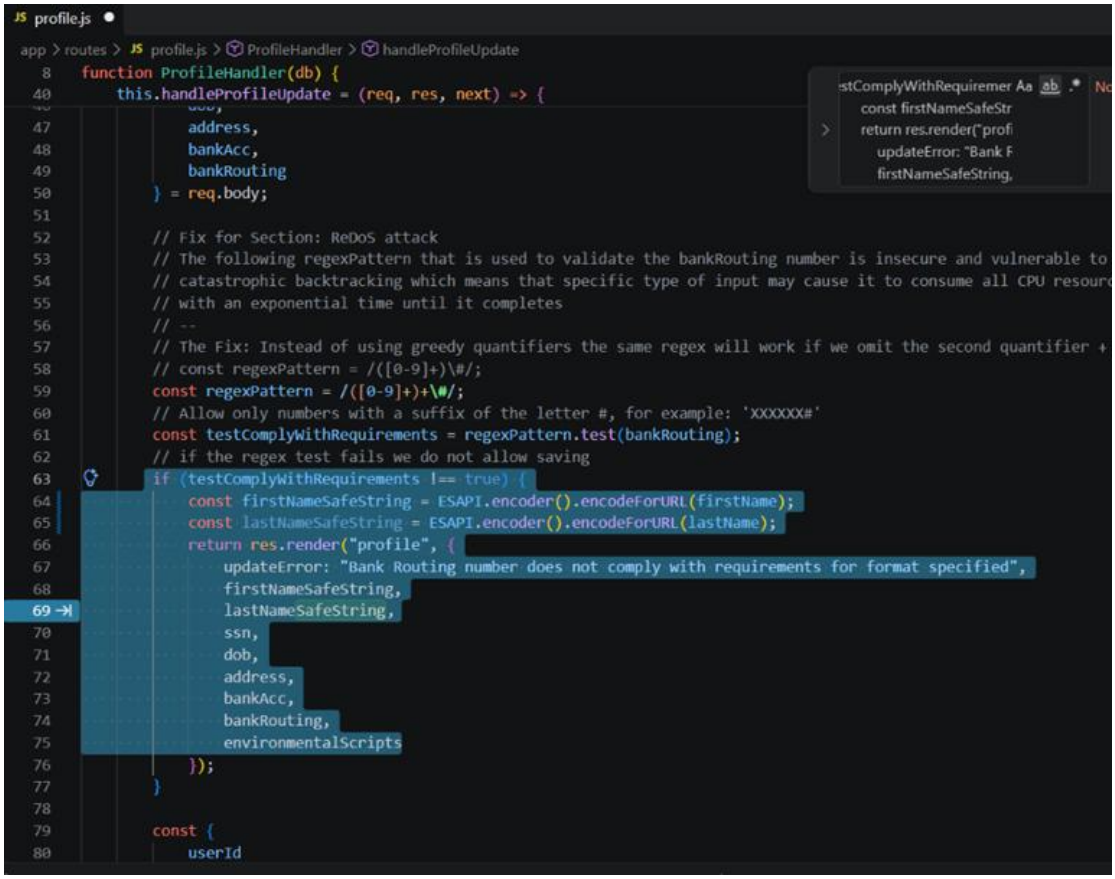
The document reports that the payload becomes stored and is rendered in the navbar username display on subsequent page loads, not only on the profile page. This establishes persistence rather than a one-time reflected result.

- handleProfileUpdate assigns firstName to firstNameSafeString without encoding in the validation-failure path.
- The unescaped value is persisted through profile.updateUser().
- The value is rendered in the profile-page href and in the navbar username display.
- swig.setDefaults({ autoescape: false }) removes the template engine's automatic escaping safety net.

### *3.3 Remediation*

The document specifies context-appropriate encoding. For the value used in an href, the documented guidance is to use URL encoding rather than treating the value as ordinary HTML text. It also documents encoding firstName and lastName before they are persisted so later rendering locations receive safe text.

The supplied implementation evidence specifically references ESAPI encoding before the database write.




Evidence 44 -Remediation code or encoding implementation. [File: image44.png]




Evidence 45 -Supporting remediation evidence. [File: image45.png]



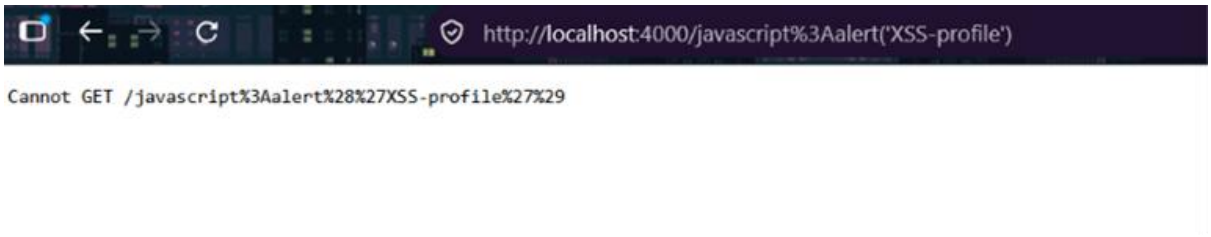

Evidence 46 -Supporting encoded-value evidence. [File: image46.png]

### *3.4 Verification after the fix*

The same XSS payload is entered again after the fix. The payload is displayed as visible text rather than executed.




Evidence 47 -Post-fix profile/application result showing the payload rendered as inert text. [File: image47.png]




Evidence 48 -Supporting post-fix navigation behavior. [File: image48.png]

The document explains that the encoded value remains visible to the user but is no longer executable. When the affected link is clicked, the browser treats the encoded value as a relative path instead of a javascript: URI, producing a normal application routing error rather than executing the payload.

The document also notes that the relevant Semgrep rules did not change because the remediation is a targeted logic-level encoding change rather than a generic anti-pattern such as eval(), MD5, or a hardcoded password.

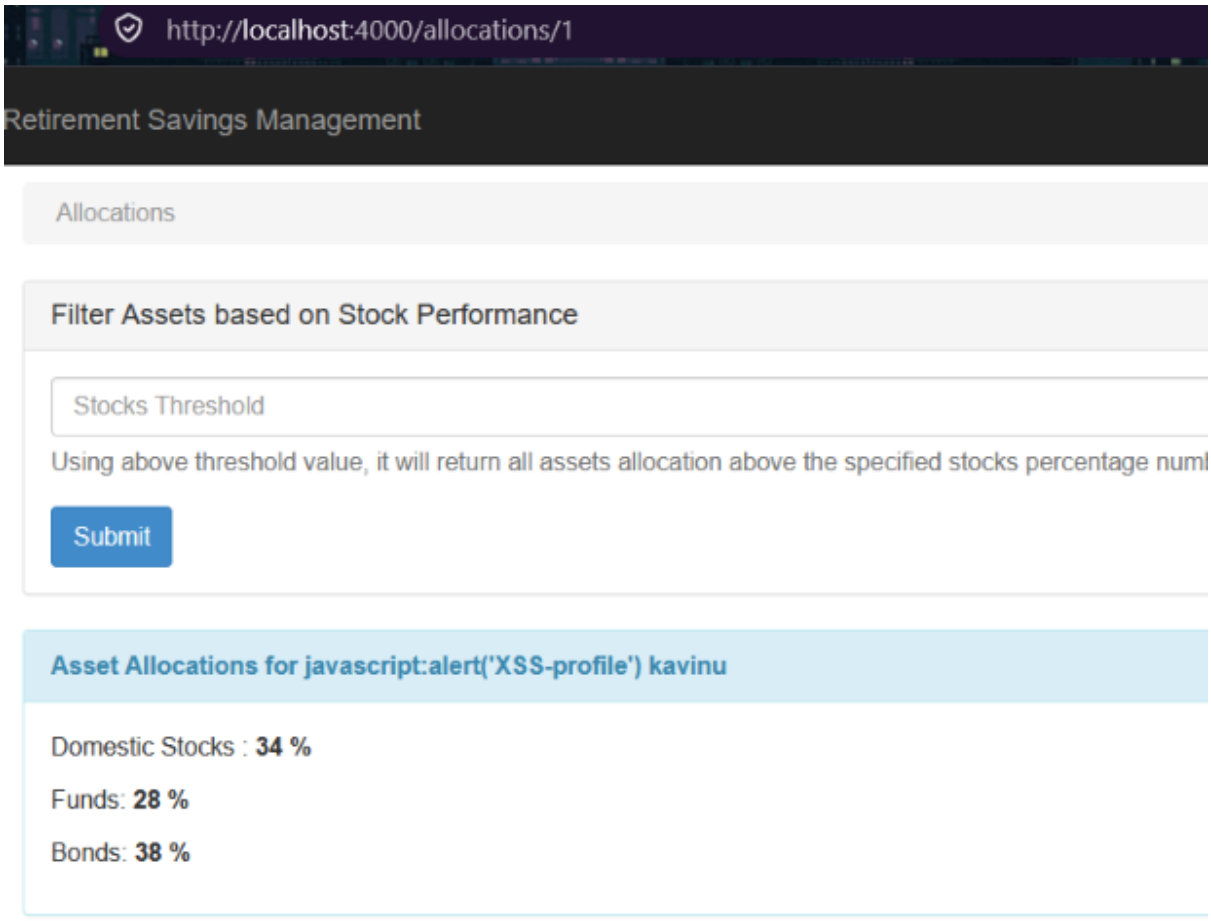
## 4. Vulnerability 4 -Insecure Direct Object Reference (IDOR)

The document identifies an authorization flaw in which userId is taken directly from the URL path parameter rather than from the authenticated session. No ownership check is performed, so changing the numeric identifier can expose another user's allocation data.

### *4.1 Initial demonstration*

The initial request uses the allocations endpoint with a numeric identifier:

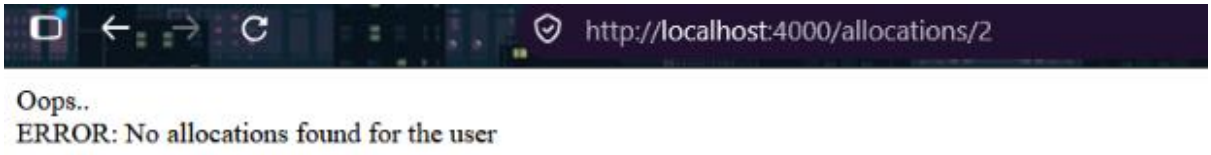
```
http://localhost:4000/allocations/1
```




Evidence 50 -Initial allocations page for the first account. [File: image50.png]

The document then checks another identifier before a second account exists.

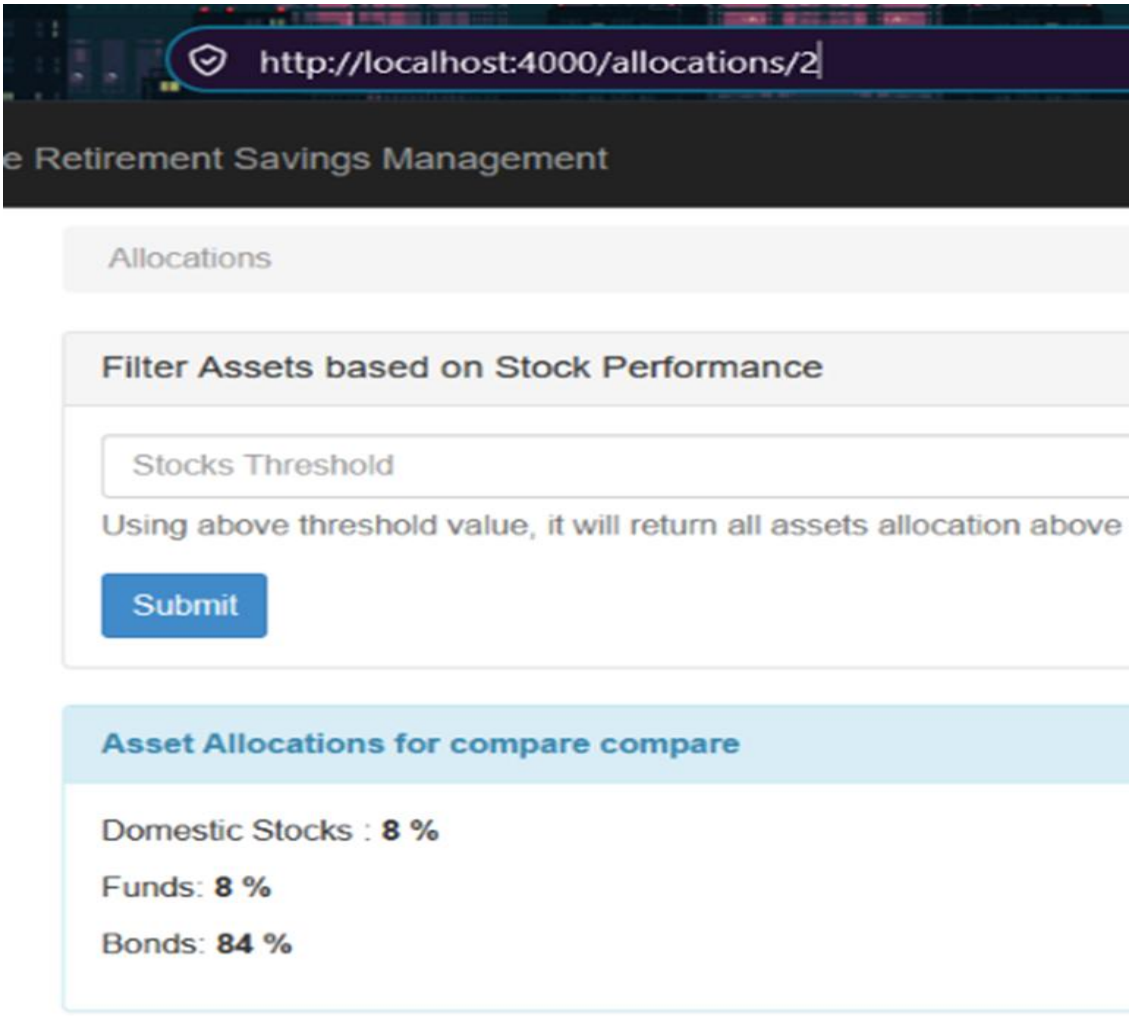
```
http://localhost:4000/allocations/2
```




Evidence 51 -Result for an identifier without a corresponding account. [File: image51.png]

### *4.2 Cross-account access demonstration*

A second account is created with the username 'compare'. The tester then logs in and requests the allocations endpoint using the second account's identifier.



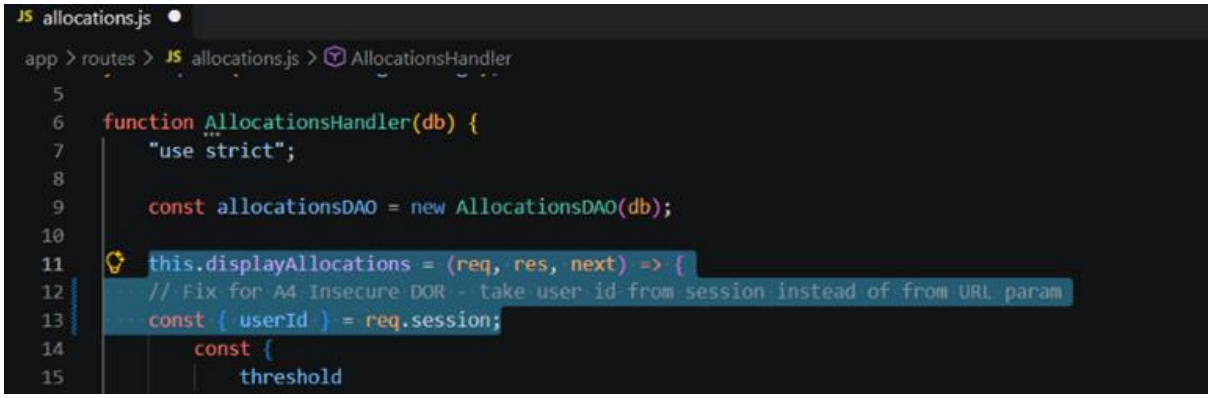

Evidence 52 -Second-account test setup or allocation evidence. [File: image52.png]

The document records the observed behavior as follows:

- /allocations/1 returned the first account's allocation data.
- /allocations/2 returned no allocations before the second account existed.
- After the second account was created, /allocations/2 returned the second account's different allocation values.
- The ability to select another user's object by changing the URL identifier demonstrates the authorization weakness.

### *4.3 Remediation*

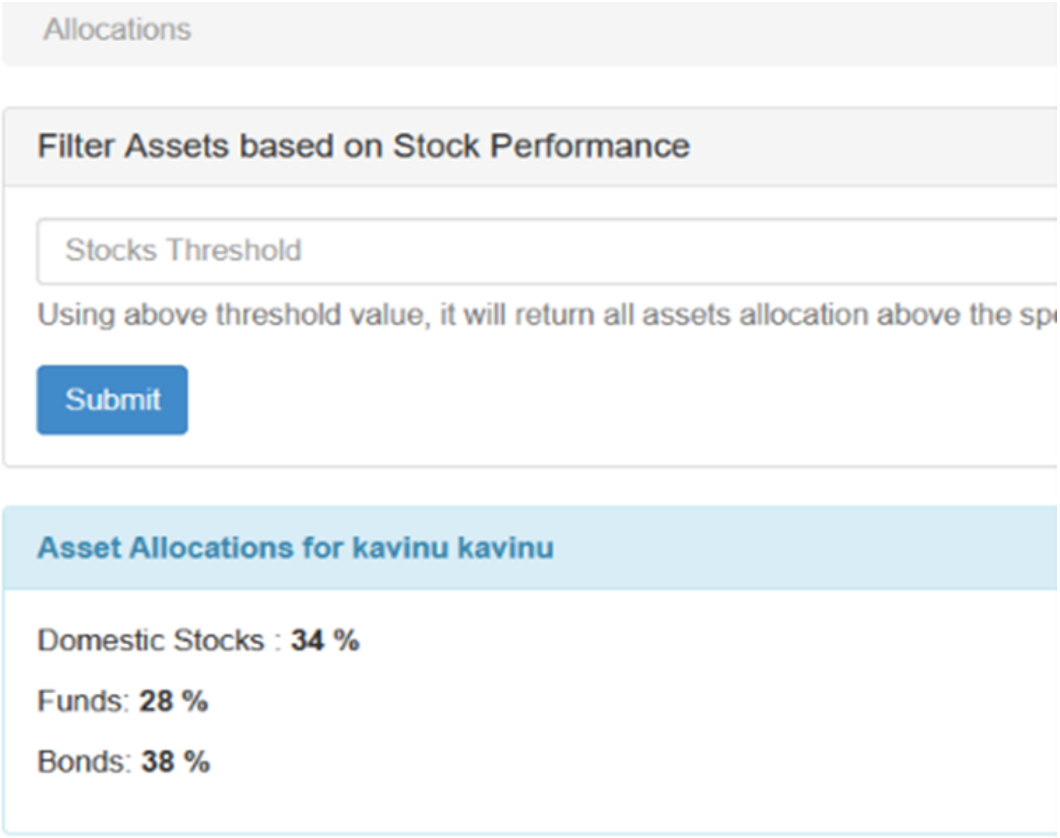
The documented fix removes the URL parameter as the source of truth for the user identity. The application should obtain userId from the server-verified authenticated session instead, so the requested allocation data is always tied to the logged-in user.




Evidence 54 -Code showing the IDOR remediation. [File: image54.png]

### *4.4 Verification after the fix*

After logging in as the first account, the tester requests /allocations/2. The expected post-fix behavior is that the application displays the authenticated user's own allocation information rather than the information belonging to account 2.




Evidence 56 -Post-fix allocation result showing the authenticated user's information. [File: image56.png]

The document states that the fix is an authorization-logic change: replacing the URL-derived identity with the authenticated session identity. As with the other targeted logic fixes, the document notes that generic Semgrep OWASP rules do not necessarily detect this kind of authorization change

## CI/CD pipeline design and security automation

 **Pipeline Architecture:**

The pipeline is implemented as a single GitHub Actions workflow, .github/workflows/ci.yml, which runs automatically on every push and every pull request. NodeGoat's inherited lint.yml and e2e-test.yml workflows were set to manual-only (workflow_dispatch), so this workflow is the only one that runs automatically.

A build job runs on an Ubuntu runner. It checks out the repository and builds the application's Docker image, tagged with the commit SHA, which confirms that every commit produces a working container. Four security jobs, secrets-scan, sast, dependency-scan and container-scan, run in parallel with it. Keeping each gate as its own job means a failure names the control that caught the problem, and each result appears as a separate check on the pull request.

 **Automated Security Gates:** Every gate returns a non-zero exit code when it finds an issue above its threshold. This fails the job and the workflow run, so the gates block a bad build rather than only warn.

| Gate | Tool | What it inspects | Configuration and fail condition |
|---|---|---|---|
| Secrets scanning | Gitleaks | Working tree and full git history | gitleaks detect --redact --exit-code 1; any leak fails; values are redacted in logs |
| SAST | semgrep | Application source code | p/javascript and p/nodejs rulesets, --error; any finding fails |
| Dependency Scanning | Npm audit | Third-party packages pinned in package-lock.json | --audit-level=high; a high or critical vulnerability fails [11] |
| Container scanning | Trivy | The built image: OS packages, npm packages and embedded secrets | --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1; a fixable high or critical vulnerability fails [12] |

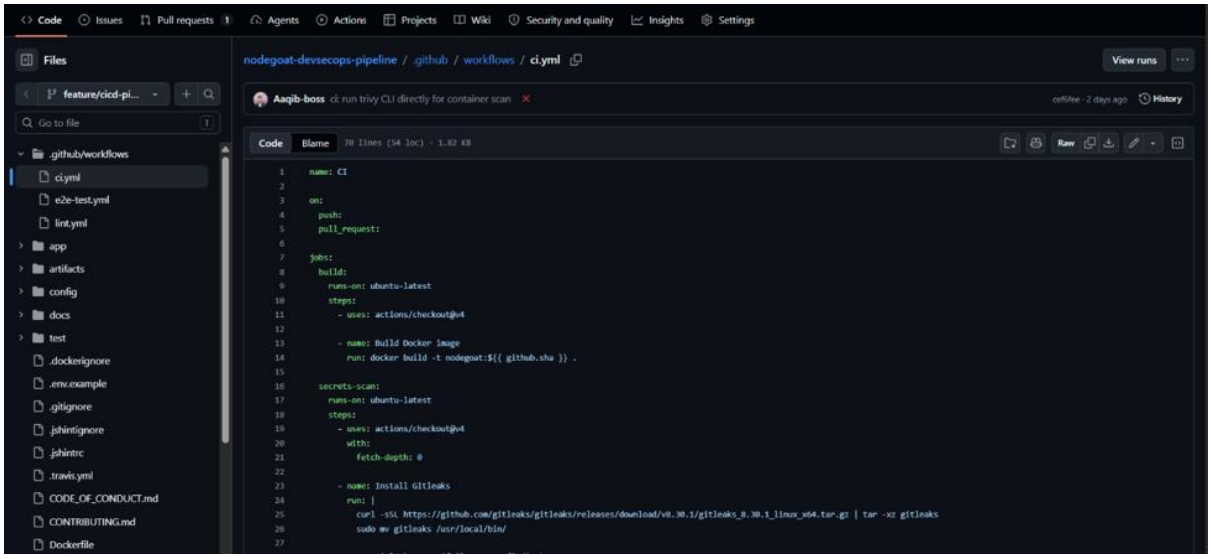
**4.3 Evidence of gates blocking a build**

The gates were first run against the unmodified NodeGoat code, and all four failed the build on real findings rather than staged ones: Gitleaks reported 3 leaked secrets (two ZAP API keys and one RSA private key), Semgrep reported 10 findings, npm audit reported 118 vulnerabilities (56 high, 30 critical), and Trivy reported 26 OS and 34 npm vulnerabilities plus one embedded secret.[11] [12]

The pipeline was merged before the remediation branches so that it evaluated each later pull request automatically. The secrets-management branch showed the Gitleaks failure clearing, and the secure-coding branch showed the Semgrep count falling, which gave consistent before/after evidence without re-running the tools by hand..

**Pipeline Failure Enforcement:** By default, most security scanners report findings but exit successfully, which lets a build pass with known problems. Each gate here is configured to exit non-zero above a defined threshold. GitHub Actions treats a non-zero exit code as a failed step, which fails the job, marks the check red on the commit and pull request, and fails the workflow run.

## 1. Pipeline overview



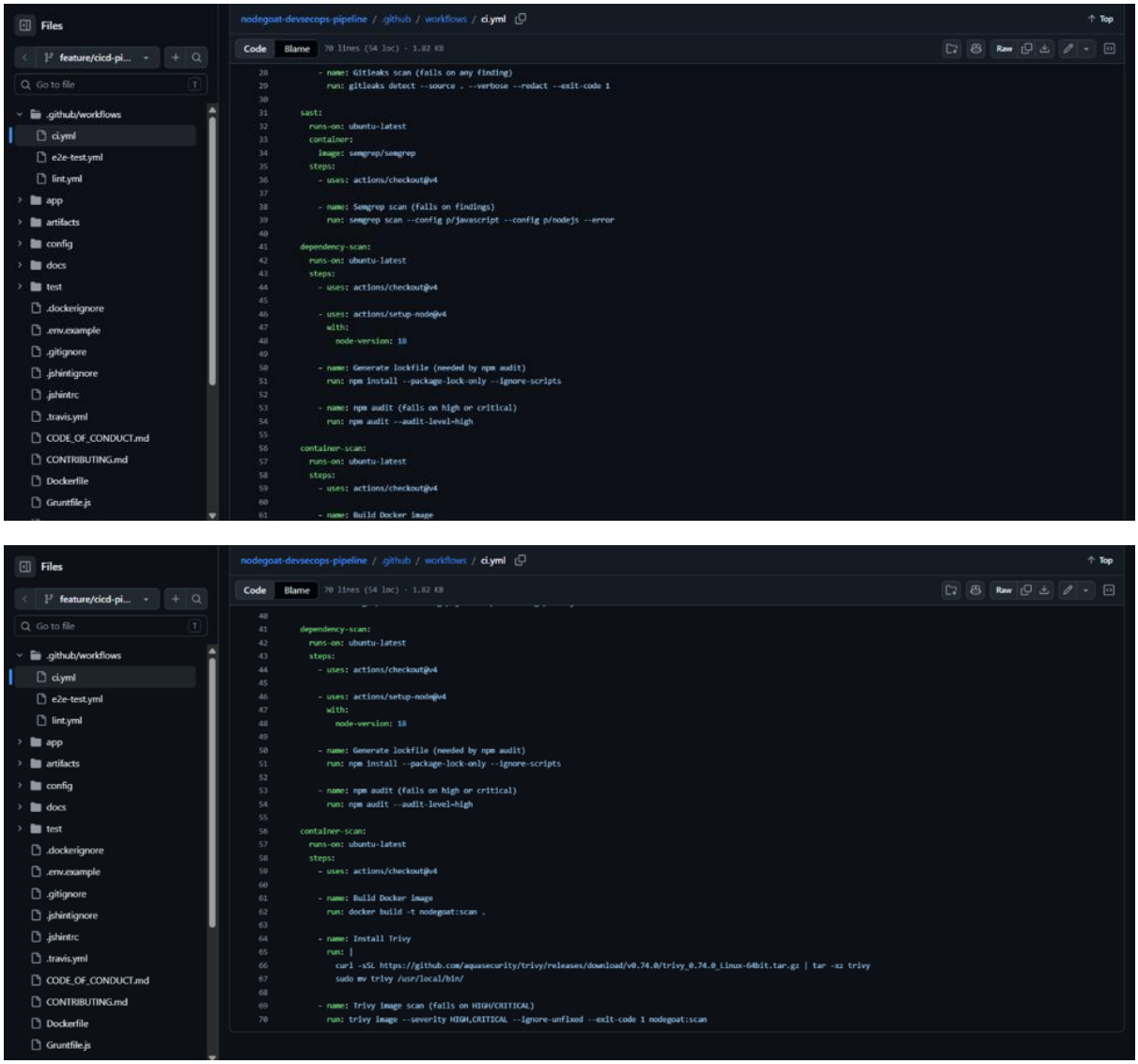






Figure 1:01a-ci-yml-file.png

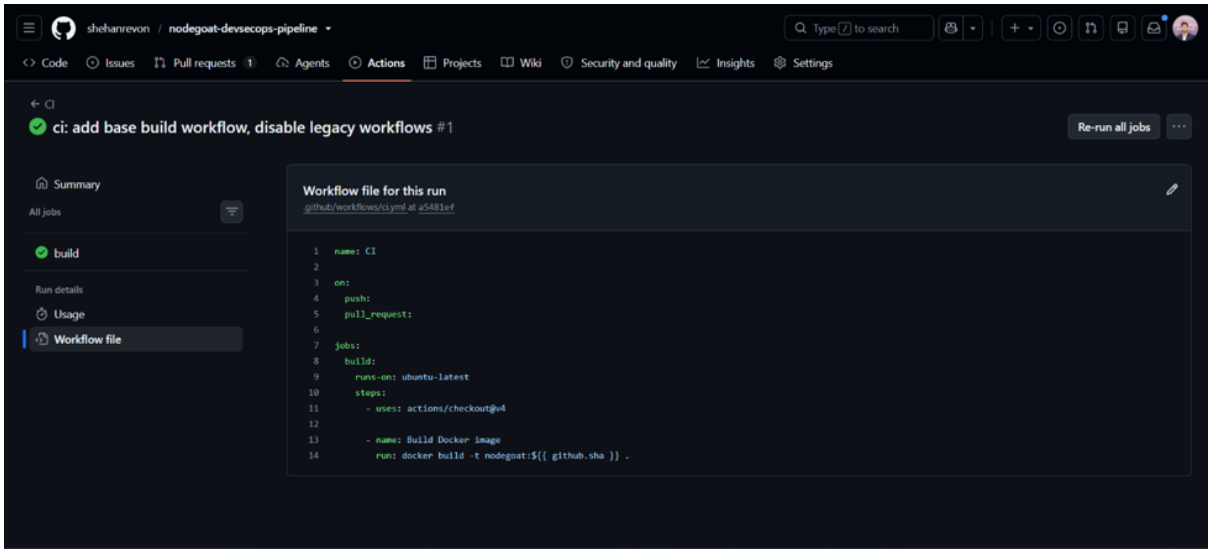



Figure 2: CI/CD Pipeline Workflow File (ci.yml)

2. Base pipeline working (proof pipeline runs)

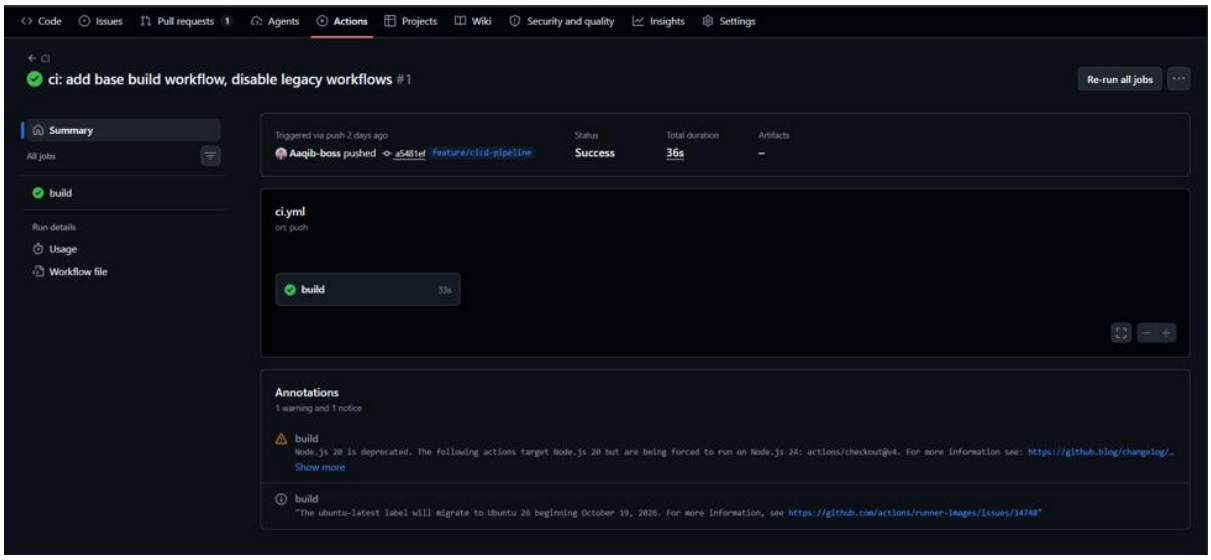



Figure 3:Base CI Pipeline -Successful Build Run

**The base CI/CD pipeline (.github/workflows/ci.yml) was first configured with a single build job that checks out the repository and builds the Docker image on every push. This confirmed the pipeline was correctly wired to GitHub Actions before the four security gates were added, with the build completing successfully in 36 seconds.**

3. Gate 1 -Secrets scanning (Gitleaks)

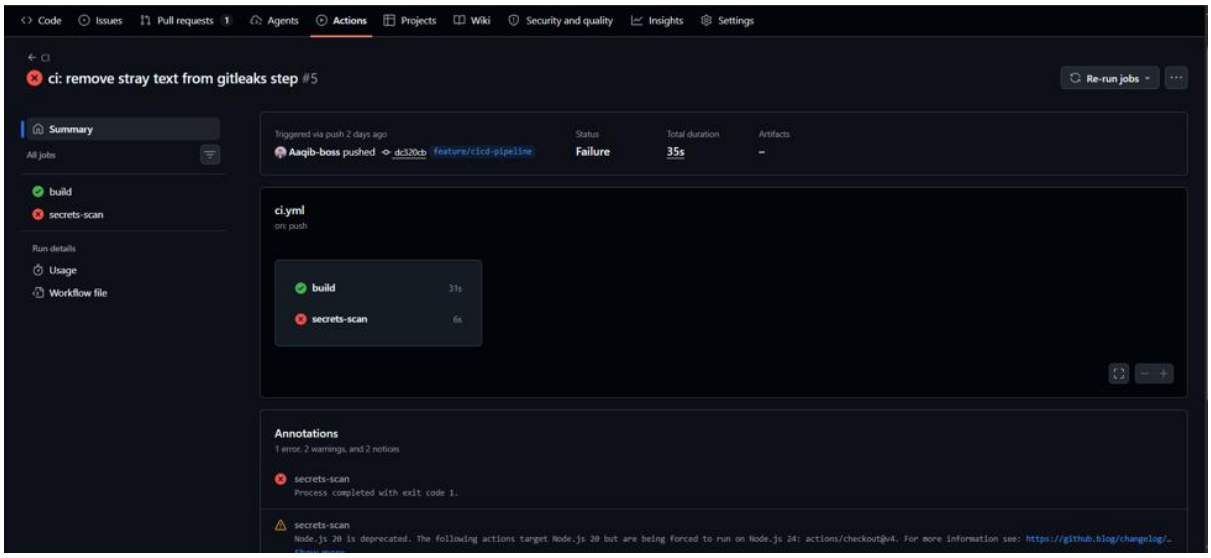
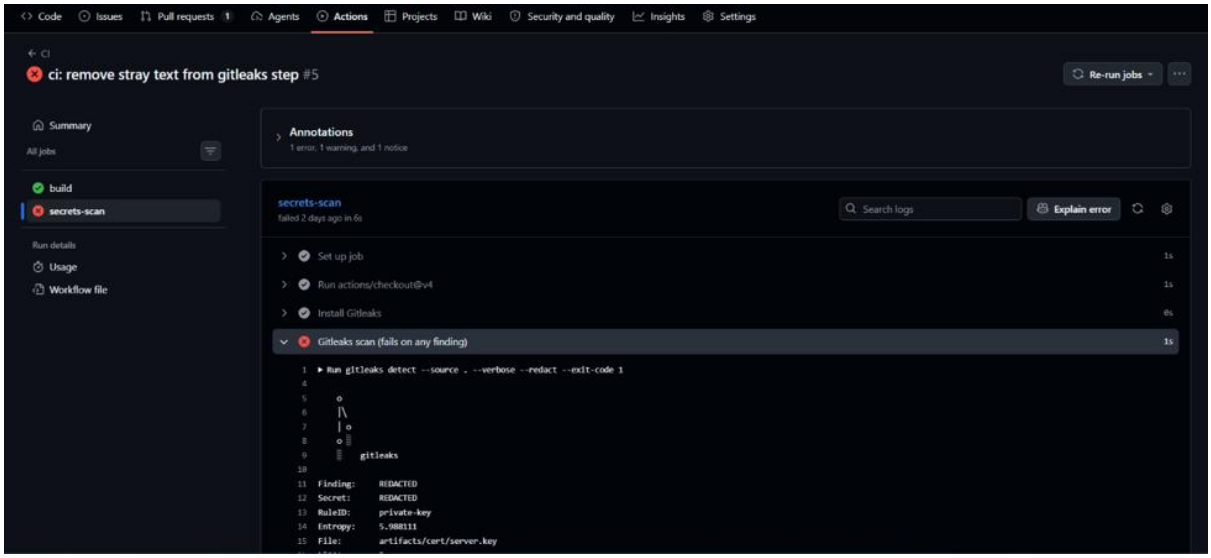
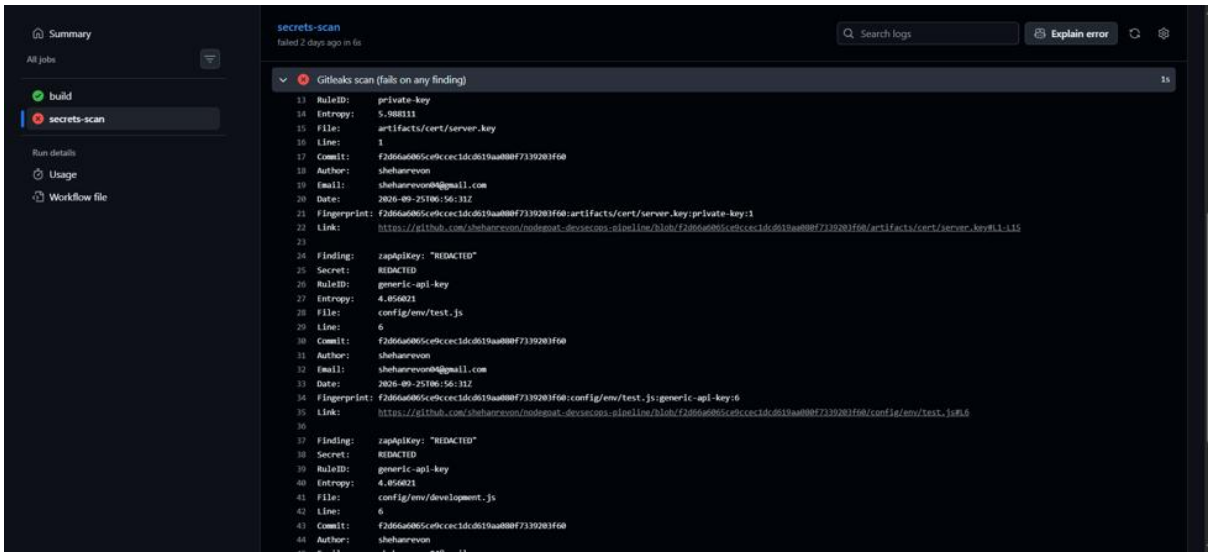



Figure 4:Secrets Scanning Gate -Pipeline Blocked

**With the Gitleaks secrets-scanning gate added, the pipeline build job still passes, but the secrets-scan job fails, demonstrating that the gate genuinely blocks the build when secrets are detected.**



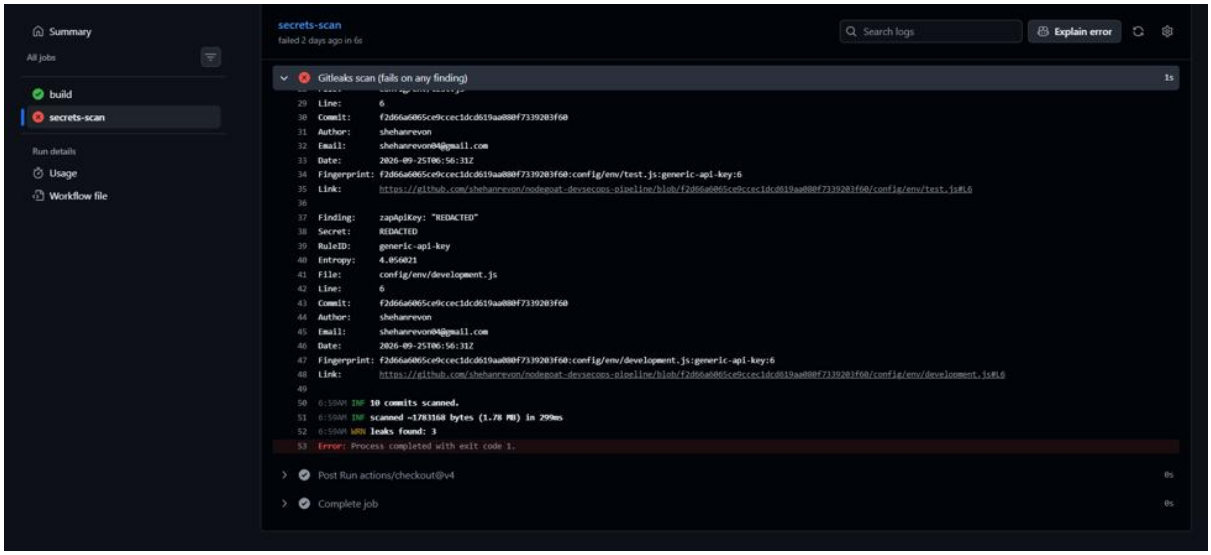



Figure 5:Gitleaks Scan Output -3 Secrets Detected

**Gitleaks scanned the full git history and detected 3 hardcoded secrets (two API keys and one RSA private key), causing the job to exit with code 1 and fail the pipeline.**

4. Gate 2 -SAST (Semgrep)

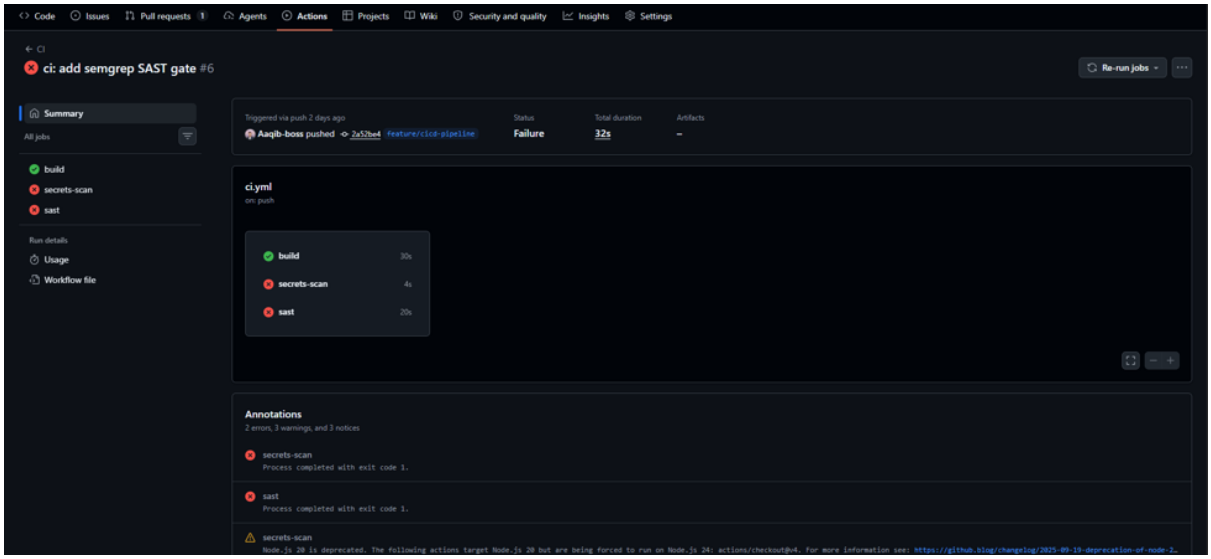



Figure 6:SAST Gate -Pipeline Blocked

**With the Semgrep SAST gate added, the sast job fails alongside secrets-scan, confirming that both gates independently block the build when issues are found.**

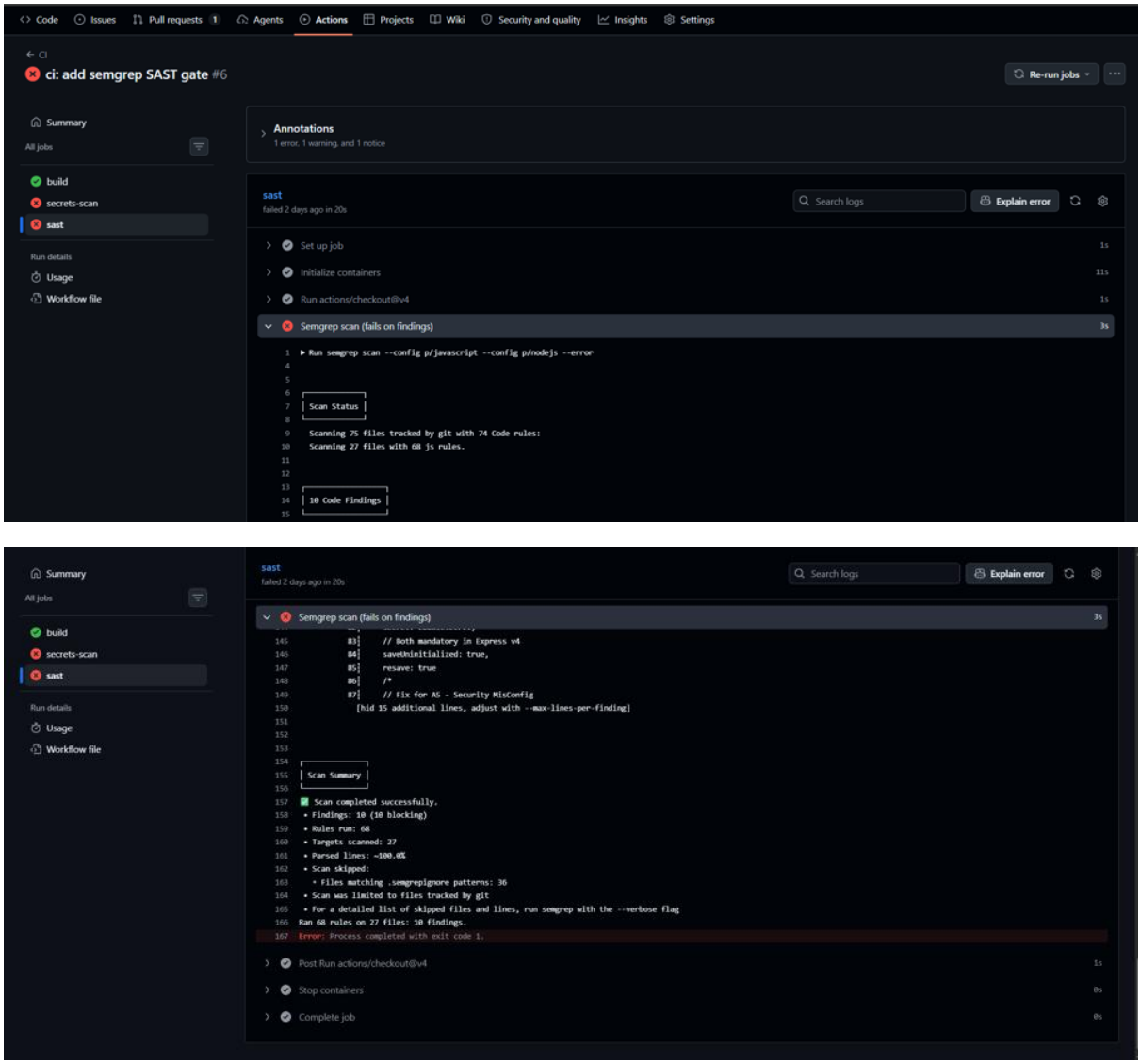



Figure 7:Semgrep Scan Output -10 Security Findings

**Semgrep scanned the application code using the javascript and nodejs rule sets and identified 10 blocking findings, including a code-injection vulnerability via eval() in contributions.js, an unvalidated open redirect, and multiple insecure session cookie settings in server.js, causing the job to exit with code 1.**

5. Gate 3 -Dependency scanning (npm audit)[11]

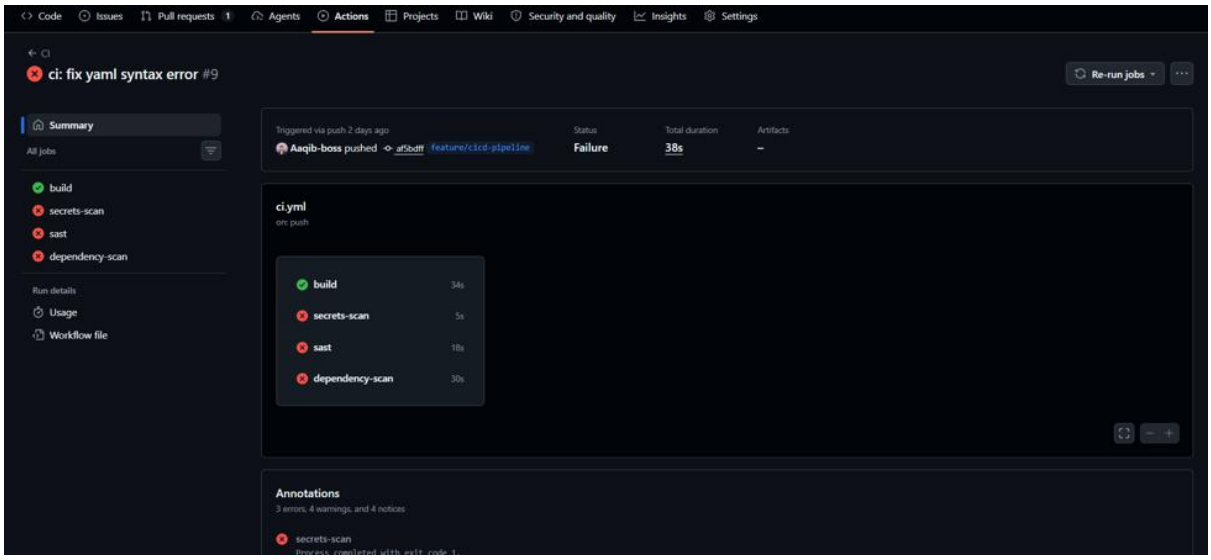



Figure 8:Dependency Scanning Gate -Pipeline Blocked

**With the npm audit dependency-scanning gate added, the dependency-scan job fails alongside the previous gates, confirming that the pipeline independently blocks builds with vulnerable dependencies.[11]**

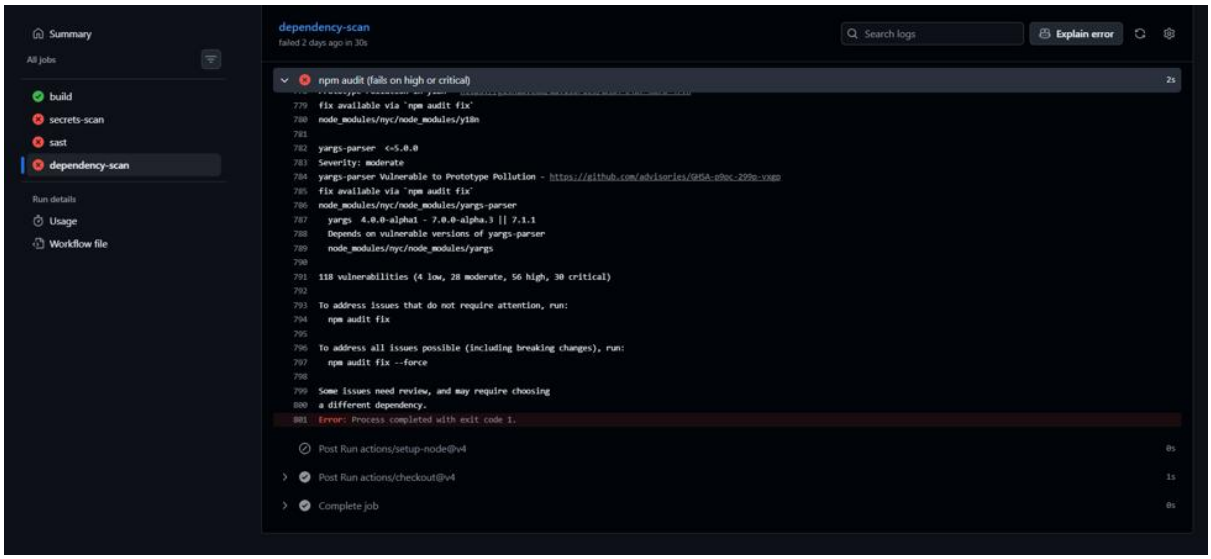



Figure 9:npm audit Output -118 Vulnerabilities Detected

**Since the repository did not include a package-lock.json, the pipeline generates one before running npm audit --audit-level=high. The scan identified 118 vulnerabilities (56 high, 30 critical) across the project's dependencies, causing the job to exit with code 1 and fail the build.[11]**

6. Gate 4 -Container scanning (Trivy)[12]

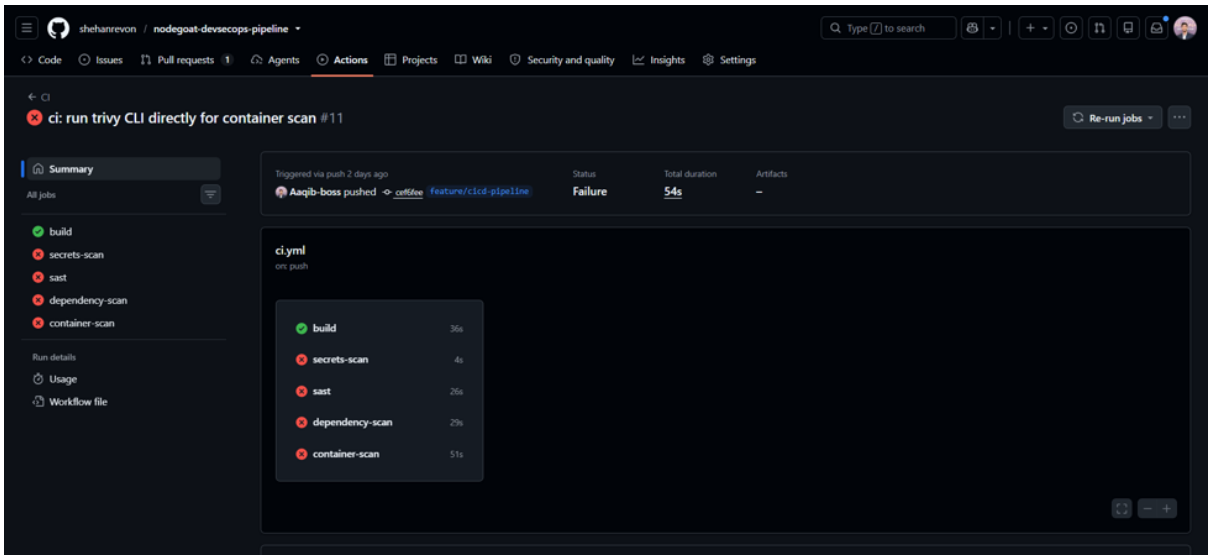
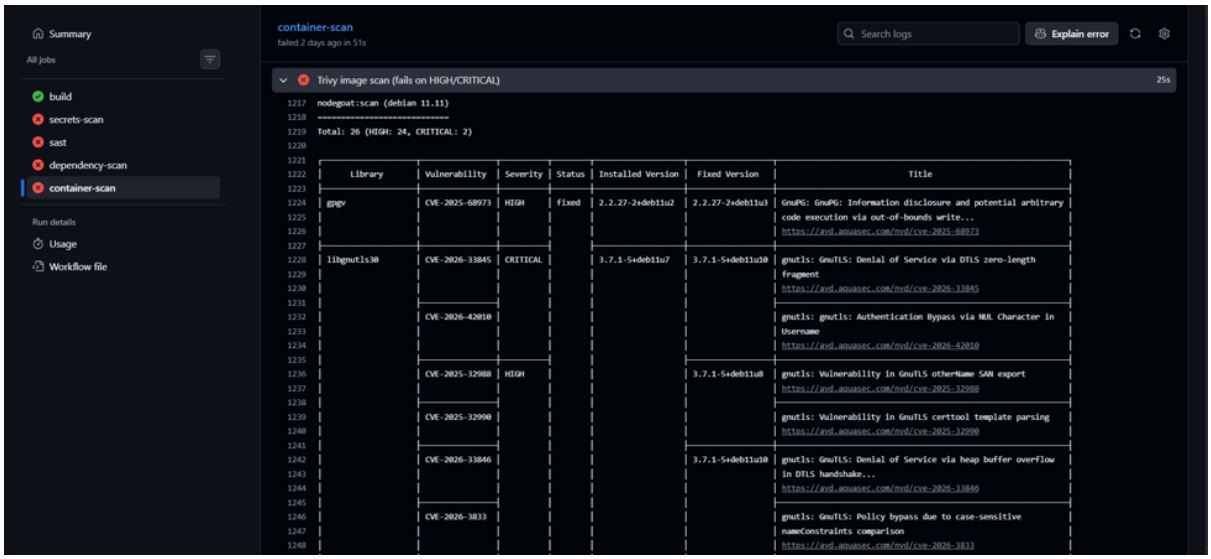



Figure 10:Container Scanning Gate -Pipeline Blocked

**With the Trivy container-image-scanning gate added, the container-scan job fails alongside all other gates, completing the four required automated security checks in the pipeline.[12]**



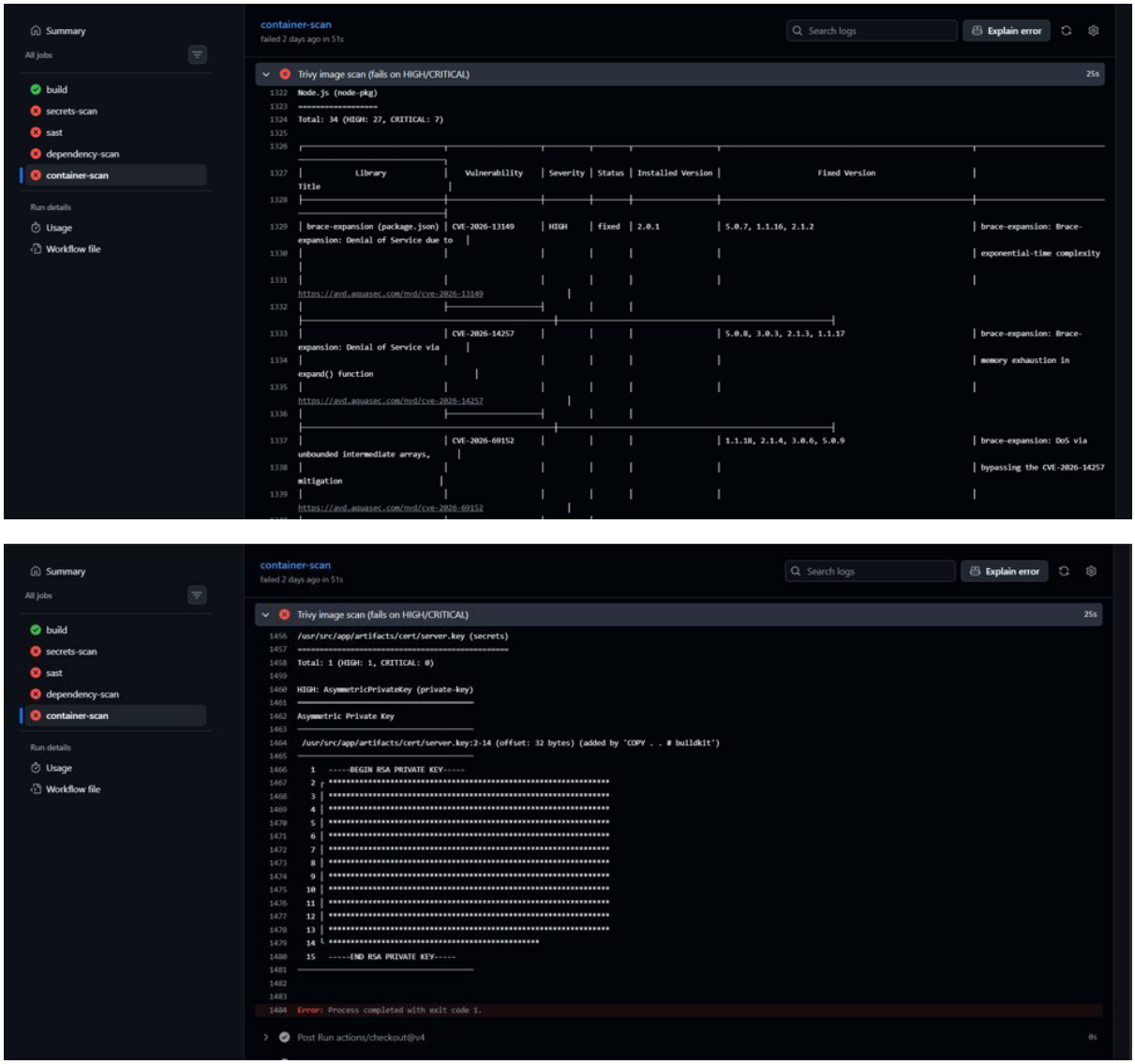






Figure 11:Trivy Scan Output -OS, Dependency and Secret Findings

**Trivy scanned the built Docker image and identified 26 HIGH/CRITICAL vulnerabilities in the base OS packages, 34 in Node.js dependencies, and 1 exposed RSA private key (artifacts/cert/server.key) embedded in the image via COPY . ., causing the job to exit with code 1.[12]**

7. Full pipeline final state

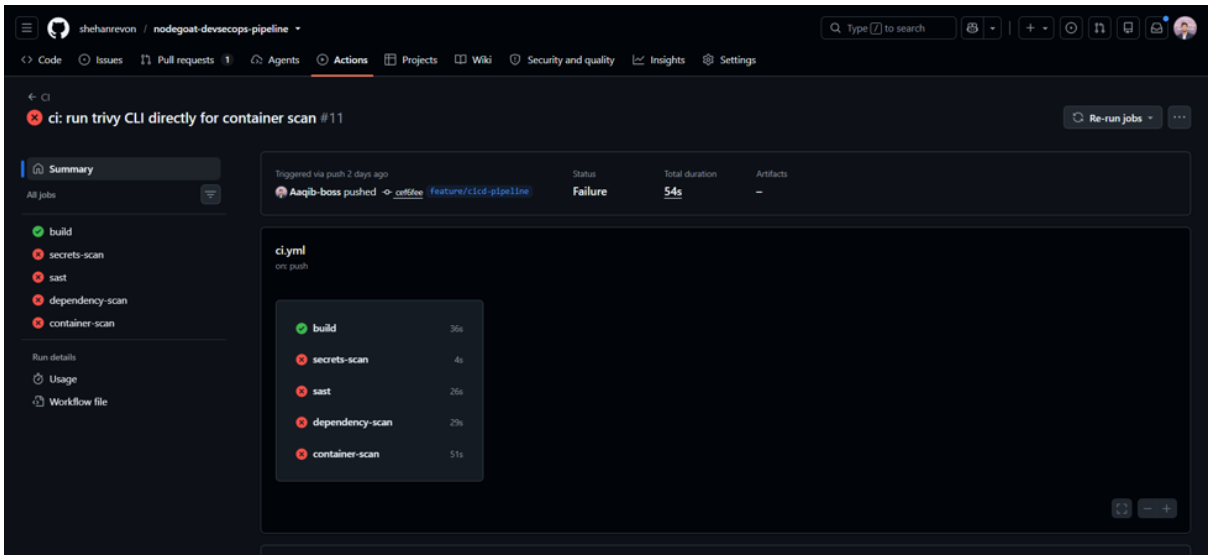



Figure 12:Final Pipeline Structure -All Four Security Gates Implemented

**The completed CI/CD pipeline consists of five jobs: a base build job (green) and four independent security gates -secrets scanning (Gitleaks), SAST (Semgrep), dependency scanning (npm audit), and container image scanning (Trivy). At this stage all four gates fail, reflecting the genuine security issues present in the unmodified NodeGoat application. This satisfies the assignment requirement that at least one gate must be capable of genuinely blocking a build.[11] [12]**

Well-designed pipeline with effective automated security checks, clear workflow and strong scan/execution evidence.

The pipeline is defined in .github/workflows/ci.yml and runs automatically on every push and pull request to main. NodeGoat's original lint.yml and e2e-test.yml were set to manual-only (workflow_dispatch) so they would not interfere. A base build job installs dependencies and builds the application and its Docker image. Four security jobs then run as independent gates. A non-zero exit code from any gate fails the run, so a bad change cannot pass unnoticed.

| Gate | Tool | Threshold/flag | Result on unmodified code |
|---|---|---|---|
| Secrets scanning | Gitleaks | --exit-code 1 (any leak) | 3 leaks (2 ZAP API keys, 1 RSA private key) |
| SAST | Semgrep | --error (any finding) | 10 findings |
| Dependency scanning | Npm audit | --audit-level=high | 118 vulnerabilities (56 high, 30 critical) |
| Container scanning | Trivy | HIGH, CRITICAL; --exit-code 1 | 26 OS + 34 npm vulnerabilities + 1 secret |

These gates blocked the build on real findings in the unmodified application, not on staged failures. Figure [X] shows the failing checks in the pull request view. Each tool addresses a different layer: Gitleaks inspects committed content, Semgrep inspects our own code, npm audit inspects third-party packages declared in package.json, and Trivy inspects the built image including the operating system layer.[11] [12]

**Pipeline as observer.** The pipeline was merged before the remediation branches so it would evaluate every later pull request automatically. The secrets branch shows the Gitleaks findings dropping, and the secure-coding branch shows the Semgrep count falling. This produced consistent before/after evidence without re-running tools by hand. **[https://github.com/shehanrevon/nodegoat-devsecops-pipeline/actions/workflows/ci.yml]**

**Reproducibility.** The dependency job originally generated a lockfile inside the pipeline. A committed package-lock.json now pins exact versions so npm audit results are identical across runs.

## Secrets management

➢ No credentials are stored in the repository. The hardcoded ZAP API keys and RSA private key found by Gitleaks were removed from the code and replaced with references to environment variables.

1. cookieSecret and cryptoKey now read from environment variables.

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content config\env\all.js
// default app configuration
const port = process.env.PORT || 4000;
let db = process.env.MONGODB_URI || "mongodb://localhost:27017/nodegoat";

module.exports = {
    port,
    db,
    cookieSecret: process.env.SESSION_SECRET || "session_cookie_secret_key_here",
    cryptoKey: process.env.CRYPTO_KEY || "a_secure_key_for_crypto_here",
    cryptoAlgo: "aes256",
    hostName: "localhost",
    environmentalScripts: []
};
```

Figure 1: config/env/all.js after remediation.

2. zapApiKey now reads from an environment variable in both config files.

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content config\env\development.js
module.exports = {
   // If you want to debug regression tests, you will need the following which is also in the test config:
   zapHostName: "192.168.56.20",
   zapPort: "8080",
   // Required from Zap 2.4.1. This key is set in Zap Options -> API _Api Key.
   zapApiKey: process.env.ZAP_API_KEY,
   // Required if debugging security regression tests.
   zapApiFeedbackSpeed: 5000, // Milliseconds.
   environmentalScripts: [
      // jshint -W101
      `<script>document.write("<script src='http://" + (location.host || "localhost").split(":")[0] + ":35729/livereload.js'></" + "script>");</script>`
      // jshint +W101
   ]
};
```

Figure 2: :development.js after remediation .

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content config\env\test.js
module.exports = {
    // If you want to debug regression tests, you will need the following.
    zapHostName: "192.168.56.20",
    zapPort: "8080",
    // Required from Zap 2.4.1. This key is set in Zap Options -> API _Api Key.
    zapApiKey: process.env.ZAP_API_KEY,
    zapApiFeedbackSpeed: 5000 // Milliseconds.
};
```

Figure 3: : test.js after remediation .

➢ **Pipeline:** Secrets required by workflows are stored as GitHub Actions encrypted secrets and read with the secrets context (${{ secrets.NAME }}). GitHub encrypts them at rest and masks them in logs, and they are never present in the checkout.[13]

1. **Encrypted GitHub Actions secrets configured for the repository.**

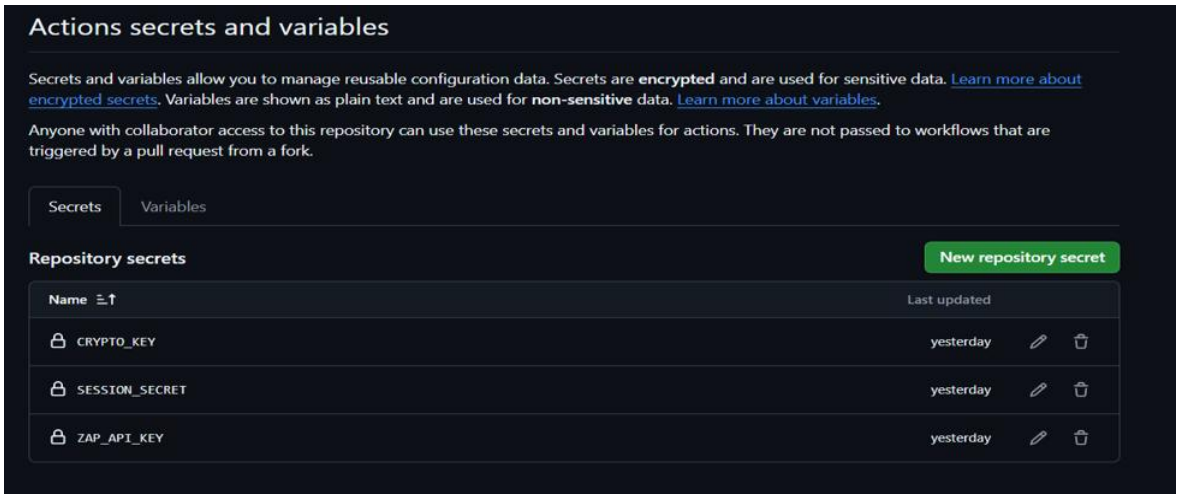



Figure 4: GitHub Actions repository secrets

➢ **Running application:** Secrets are passed to containers as environment variables through docker-compose.yml, read from a local .env file. .env is listed in .gitignore, and a placeholder .env.example documents the required variables without values.

1. Secrets passed into the web container via the environment: block.

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content docker-compose.yml
services:
  mongo:
    image: mongo:6
    container_name: nodegoat-mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    networks:
      - nodegoat-net

  web:
    build: .
    container_name: nodegoat-web
    depends_on:
      - mongo
    environment:
      - MONGODB_URI=mongodb://mongo:27017/nodegoat
      - SESSION_SECRET=${SESSION_SECRET}
      - ZAP_API_KEY=${ZAP_API_KEY}
      - CRYPTO_KEY=${CRYPTO_KEY}
      - NODE_ENV=development
      - PORT=4000
    ports:
      - "4000:4000"
    networks:
      - nodegoat-net

networks:
  nodegoat-net:
    driver: bridge

volumes:
  mongo-data:
```

Figure 5: docker-compose.yml environment configuration.

2. Placeholder values in .env.example; secret files excluded via .gitignore.

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content .env.example
SESSION_SECRET=changeme_in_local_env_only
ZAP_API_KEY=your_zap_api_key_here

CRYPTO_KEY=your_crypto_key_here
```

Figure 6: .env.example configuration.

```
PS C:\dev\nodegoat-devsecops-pipeline> Get-Content .gitignore
# Node modules
node_modules

# Logs and databases
*.log
*.db

# OS files
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
Icon?

# Idea stuff
.idea/**

# Zap output
report*.html

# e2e
test/e2e/screenshots/
test/e2e/videos/

# ignore sensitive files
.env.local
.env

# ignore Snyk Code scanner files
.dccache

artifacts/cert/*.key
artifacts/cert/*.crt
gitleaks-report.json
 cookies.txt
```

Figure 7: .gitignore configuration.

3. Runtime logs confirming secrets resolve to real values, not undefined.

```
PS C:\dev\nodegoat-devsecops-pipeline> docker compose down
[+] down 3/3
 ✓ Container nodegoat-web                              Removed
 ✓ Container nodegoat-mongo                            Removed
 ✓ Network nodegoat-devsecops-pipeline_nodegoat-net    Removed
```

Figure 8: docker compose down.

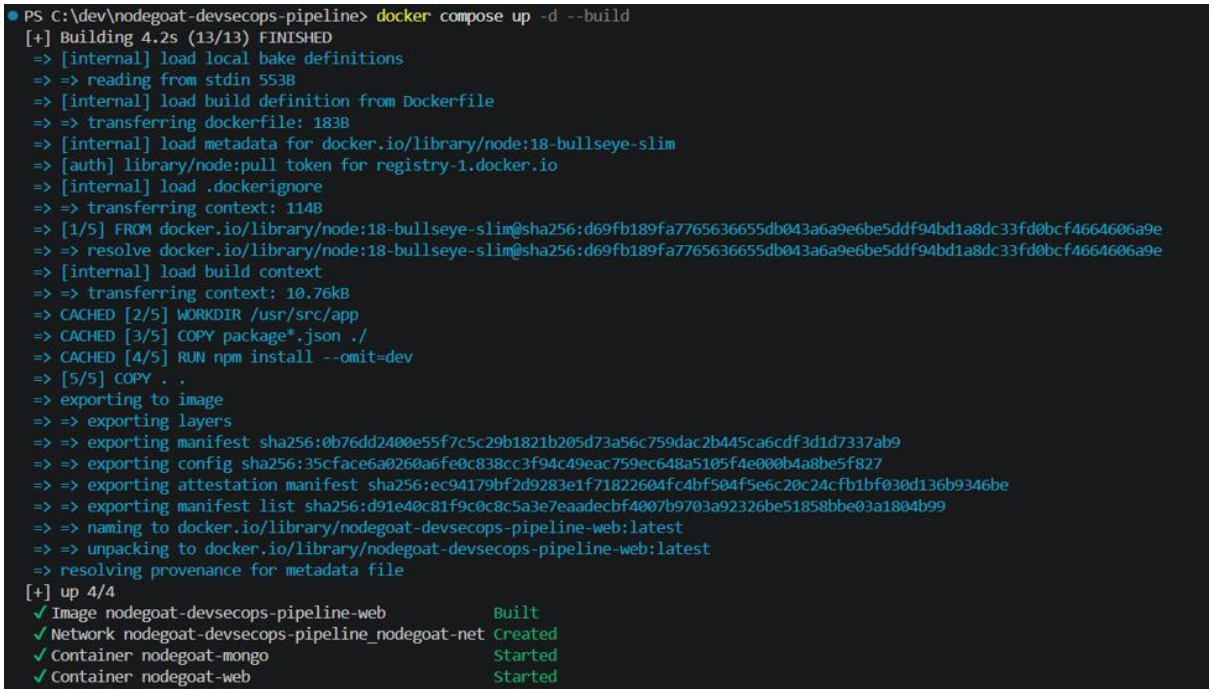



Figure 9: docker compose up --build.

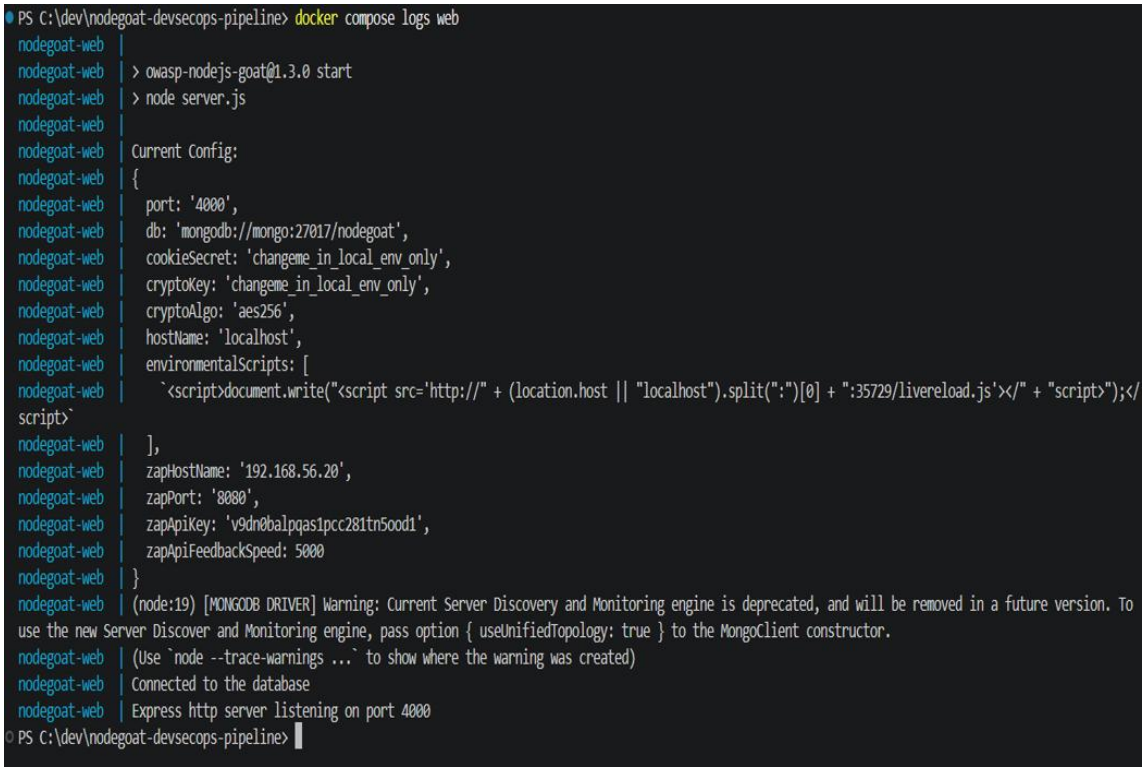



Figure 10: docker compose logs web showing resolved secrets.

➢ **Verification:** Gitleaks runs on every push, and the final scan of main reports no leaks

1. **Gitleaks scan run after remediation, still showing 3 findings**

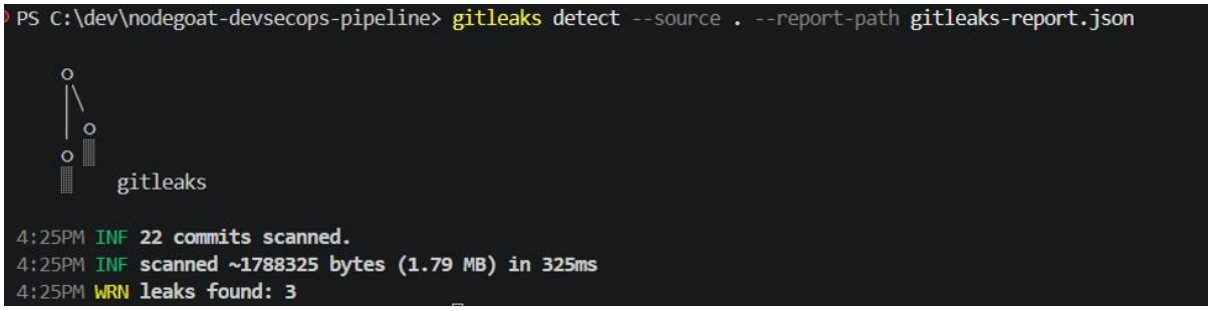



Figure 11: Gitleaks scan following remediation.

2. **All 3 findings trace back to the original baseline commit, not current code**

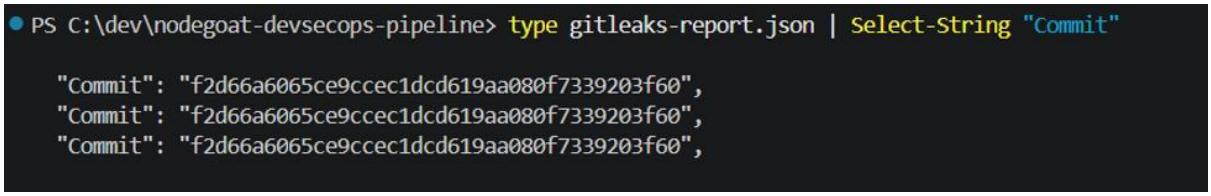



Figure 12: Findings confirmed historical (commit f2d66a6)

3. **Commit history for the secrets management workstream.**

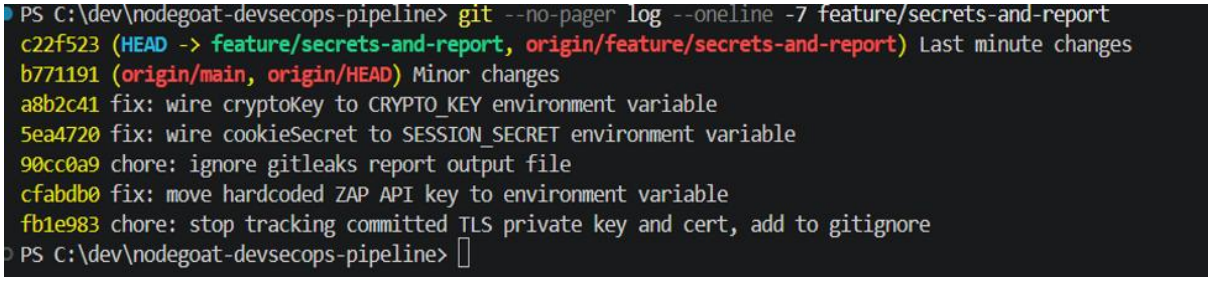



Figure 13: Commits on feature/secrets-and-report

## Industry trends and case study analysis

- **Case Study:**

**Codecov supply chain attack (2021)**

Codecov provides a Bash Uploader script that CI pipelines download and run to send code-coverage reports. Between 31 January and 1 April 2021 an attacker modified this script without authorisation, and the attacker obtained the credentials for the change by extracting them from a Docker image. The added line sent environment variables from customers' CI runners, including API keys and tokens, to a remote server. A customer noticed the compromise only when the script's hash differed from the published one. Codecov advised customers to re-roll every credential in their CI environment variables. Bash Uploader Security Update - Codecov.io +5

The incident shows two DevSecOps lessons. First, the CI environment is a high-value target because it concentrates secrets, and trusting an unpinned third-party script hands those secrets to whoever controls it. Second, a secret baked into a build artefact (here a Docker image) is effectively public to anyone who can pull the artefact.

Our project applies the same lessons in three ways. Secrets are exposed only to the jobs that need them and are never committed, which limits what a compromised step could steal. Gitleaks and Trivy's secret scanning detect credentials left in source and in images. Trivy and npm audit monitor dependencies. We acknowledge our gates would not have detected the Codecov attack itself, since it was a tampered vendor script; pinning actions to commit SHAs and verifying checksums would be the appropriate defence (see Section 7).[11] [12]

# Reflection and future improvements

### 7.1 Project reflection

**Challenges and lessons.**

- **Compatibility** - NodeGoat's original mongodb driver (^2.1.18) was incompatible with a modern MongoDB server, so session.js and related files needed updates before the application would run. We fixed this as a separate pull request. Keeping compatibility work apart from vulnerability fixes made it clear later which changes counted as security remediation.
- **Merge order.** A pipeline that fails on the unmodified code cannot be merged onto a protected branch without a plan. We merged the pipeline first so it could observe every later pull request, then the secrets and secure-coding branches, and this produced consistent before/after evidence. The lesson is that in DevSecOps the order in which controls are introduced affects what you can prove.
- Reproducibility. The dependency scan depended on a lockfile the repository did not have. Committing package-lock.json gave stable, repeatable audit results.
- **Secrets and history.** Removing a secret from the current files does not remove it from git history, and Gitleaks scans that history. Preventing secrets from entering the repository is far cheaper than cleaning up afterwards.
- **Measurement.** Before/after SAST results are only comparable if the same command and rulesets are used both times. Any change to the command changes the counts.
- **Coordination.** With four contributors and one gatekeeper, small overlaps (such as the same compatibility fix appearing on two branches) had to be spotted in code review.

**Overall lesson.** Automated gates are only useful when they are configured to fail, run on every change, and produce evidence that a person can inspect.

### 7.2 Future improvements

1. **Required status checks on main**. Branch protection currently requires a pull request. Once remaining findings are triaged, requiring the four gate checks would enforce them at merge time.
2. **Harden the pipeline itself**. Pin GitHub Actions to commit SHAs instead of version tags, verify checksums for the Gitleaks and Trivy binaries downloaded in the workflow, and set permissions: contents: read so a compromised step has minimal access. The Codecov case shows why this matters.[12] [14]
3. **Runtime secret injection with HashiCorp Vault.** This would replace `.env` files, allow secret rotation, and provide an audit trail.
4. **IaC and Dockerfile scanning**. Trivy's configuration scanner or Checkov could check the Dockerfile and docker-compose.yml for issues such as the published MongoDB port and containers running as root.[12]
5. **Container hardening.** Run as a non-root user, use a slimmer or distroless base image, upgrade from Node 18, enable MongoDB authentication, and stop publishing port 27017 to the host.
6. **Supply chain assurance.** Generate an SBOM (for example with Syft) and sign images with cosign so consumers can verify what was built.
7. **DAST and pre-commit checks.** Make the OWASP ZAP scan a regular pipeline stage, and add a Gitleaks pre-commit hook so secrets are stopped before they reach GitHub.
8. **Longer term.** If the application were deployed to Kubernetes, the next steps would be network policies, pod security standards and admission control. Runtime detection with a tool such as Falco would add monitoring, which none of the current static gates provide.

## Contribution statement

| Group member | Role | Specific contributions |
|---|---|---|
| Kosgollage S R - IT24102747 | Lead & Architecture | Managed repository baseline, PR reviews, and merges. Fixed MongoDB driver compatibility issues. Configured Docker containerization. Drafted the architecture diagram, STRIDE threat model, risk ratings, and threat-to-control mapping. |
| Kavinu Palugaswewa - IT24201195 | Exploit & Secure Coding | Identified and demonstrated 4+ application-specific vulnerabilities. Implemented secure coding fixes, successfully re-tested the exploits to prove remediation, and generated before/after Semgrep (SAST) finding diffs. |
| Aaqib H.M - IT24102514 | CI/CD & Security Gates | Designed the GitHub Actions pipeline (ci.yml). Integrated and configured four automated security gates (Gitleaks, Semgrep, npm audit, Trivy) with strict thresholds to genuinely fail the build. Captured pipeline failure evidence. |
| Shaheen M.F.M - IT24102899 | Secrets Management | Audited the codebase to remove hardcoded credentials (e.g., zapApiKey, RSA private keys). Configured and integrated GitHub Actions encrypted secrets to securely provision credentials at runtime. |

## AI usage disclosure

| Prompt | Justification |
|---|---|
| "How to update the Node.js MongoDB driver from version ^2.1.18 to ^3.6.0 and resolve connection errors in session.js?" | To resolve legacy compatibility issues between the unmodified NodeGoat application and a modern MongoDB container during the initial baseline setup. |
| "How to configure a Trivy GitHub Actions step to exit with code 1 specifically when HIGH or CRITICAL vulnerabilities are found?" | To correctly configure the container scanning security gate to genuinely fail the pipeline build, meeting the automated enforcement requirement. |
| "What are the most common STRIDE threats applicable to an Express.js backend connected to a MongoDB database?" | To brainstorm and validate realistic, application-specific attack vectors for the system architecture and threat modelling assessment. |
| What is the correct syntax for mitigating NoSQL injection when querying MongoDB in a Node.js application?" | To identify the proper secure coding patterns (e.g., input sanitization and parameterization) required to fix the demonstrated exploits for the remediation evidence. |
| "How to pass GitHub Actions encrypted secrets as environment variables into a docker-compose up command during a CI workflow?" | To ensure database credentials and API keys are dynamically injected at runtime without hardcoding them into the repository history. |
| "How to configure npm audit to only fail a CI build on 'high' severity issues while ignoring 'low' or 'moderate' warnings?" | To establish a realistic threshold for the dependency scanning gate that blocks critical supply chain risks without breaking the build on minor issues. |

## References


[1] OWASP Foundation, “NodeGoat,” OWASP, GitHub repository. Available: https://github.com/OWASP/NodeGoat

[2] OWASP Foundation, “Threat Modeling Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

[3] MITRE, “CWE-943: Improper Neutralization of Special Elements in Data Query Logic,” CWE. Available: https://cwe.mitre.org/data/definitions/943.html

[4] MITRE, “CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (Eval Injection),” CWE. Available: https://cwe.mitre.org/data/definitions/95.html

[5] OWASP Foundation, “Session Management Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[6] MITRE, “CWE-384: Session Fixation,” CWE. Available: https://cwe.mitre.org/data/definitions/384.html

[7] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[8] MITRE, “CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting),” CWE. Available: https://cwe.mitre.org/data/definitions/79

[9] MITRE, “CWE-639: Authorization Bypass Through User-Controlled Key,” CWE. Available: https://cwe.mitre.org/data/definitions/639.html

[10] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1,” NIST SP 800-218, National Institute of Standards and Technology, Feb. 2022. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[11] npm, “npm-audit,” npm Documentation. Available: https://docs.npmjs.com/cli/v11/commands/npm-audit/

[12] Aqua Security, “Trivy Documentation: Container Image Scanning,” Trivy. Available: https://trivy.dev/docs/latest/target/container_image/

[13] GitHub, “Secrets,” GitHub Actions Documentation. Available https://docs.github.com/en/actions/concepts/security/secrets

[14] GitHub, “Secure use reference,” GitHub Actions Documentation. Available: https://docs.github.com/en/actions/reference/security/secure-use

[15] Codecov, “Post-Mortem / Root Cause Analysis (April 2021),” Apr. 2021. Available: https://about.codecov.io/apr-2021-post-mortem/